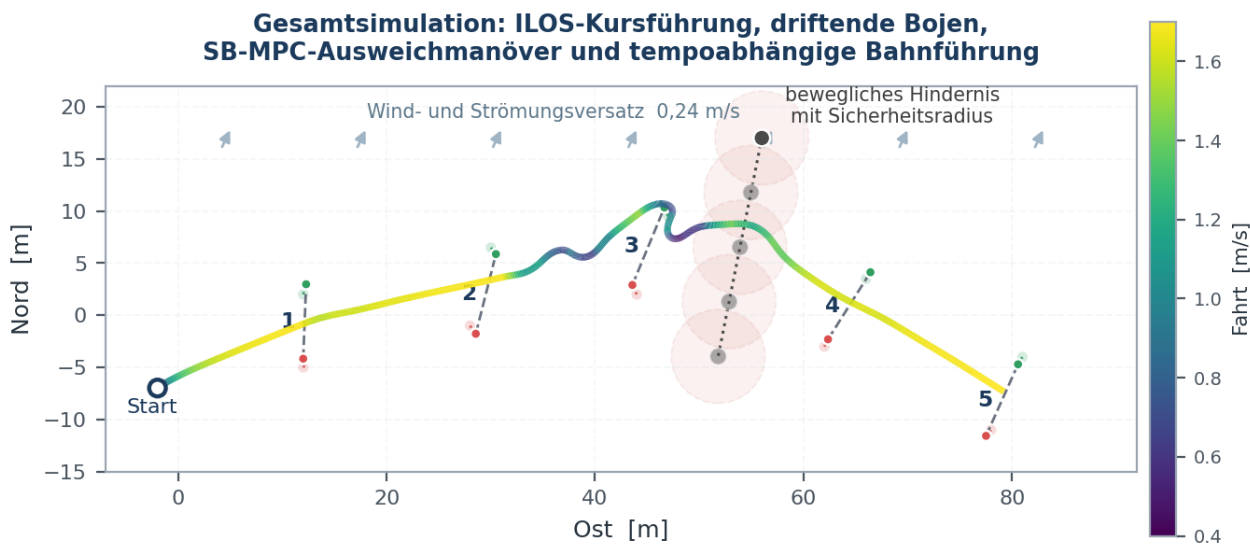


Autonome Fahrt auf einem Bojenkurs

Guidance, Navigation und Control für ein 1 m langes Boot mit zwei Motoren
– bei Wind, Wellen, driftenden Bojen und beweglichen Hindernissen

Umsetzung als eigener ROS-2-Stack

Gegenstand	Nachrüstung eines fertigen Rumpfes (ca. 1 m) mit Antrieb, Sensorik und autonomer Steuerung
Antrieb	zwei Motoren, differentielle Lenkung (skid steering)
Aufgabe	vorgegebenen Bojenkurs abfahren, dabei beweglichen Hindernissen ausweichen
Einsatzort	offenes Wasser – Wind und Seegang sind Auslegungsfall, nicht Störfall
Software	eigener ROS-2-Stack auf Linux, harte Echtzeit auf einem Mikrocontroller über micro-ROS
Inhalt	Architektur, Knotenaufbau, Hardwareauswahl, Verfahren, Auslegungsrechnungen, Teststufen, Parameter, Ablaufplan



21 Abbildungen, davon 6 aus eigenen Simulations- und Modellrechnungen. Alle Zahlenwerte sind Auslegungs-Startwerte und am realen Boot nachzumessen.

Inhalt

1. Kurzfassung und Empfehlung	5
1.1 Die sechs Kernentscheidungen	5
1.2 Was am Ende dabei herauskommt	5
1.3 Realistische Erwartungshaltung	5
2. Aufgabenstellung und Randbedingungen	6
2.1 Was das System leisten muss	6
2.2 Die Umgebung ist der eigentliche Gegner	6
2.3 Physikalische Grenzen des Rumpfes	7
2.4 Anforderungsliste	7
3. Systemarchitektur	8
3.1 Warum Schichten und nicht ein einziges Netz	8
3.2 Takt- und Latenzbudget	9
4. Softwarearchitektur mit ROS 2	10
4.1 Die Grundentscheidung: Linux allein genügt nicht	10
4.2 micro-ROS als Brücke	10
4.3 Der Knotengraph	11
4.4 Sechs Dinge, die man beim eigenen Stack falsch machen kann	12
4.5 Was Sie nicht selbst schreiben sollten	12
5. Bootsmodell und Aktorik	13
5.1 Bezugssysteme und Zustandsgrößen	13
5.2 Das Nomoto-Modell für die Drehbewegung	14
5.3 Schubaufteilung	14
5.4 Systemidentifikation am realen Boot	14
6. Hardware: Auslegung und Auswahl	16
6.1 Wie viel Schub wird tatsächlich gebraucht?	16
6.2 Motoren und Fahrtregler	16
6.3 Sensorik – hier entscheidet sich der Erfolg	17
6.4 Rechner	18
6.5 Energieversorgung	18
6.6 Stückliste als Ausgangspunkt	20
7. Zustandsschätzung und Wellenfilterung	21
7.1 Aufbau der Filterkette	21
7.2 Was der Filter leisten muss	21
7.3 Wellenfilterung: der wichtigste Kunstgriff bei Seegang	21
8. Wahrnehmung: Bojen und Hindernisse	24
8.1 Erkennung im Bild	24

8.2 Entfernungsschätzung – die eigentliche Schwierigkeit	24
8.3 Verfolgung und Zuordnung	25
8.4 Vom Bojenpaar zum Pfad	25
9. Kursführung (Guidance)	27
9.1 Line-of-Sight als Grundverfahren	27
9.2 Warum einfaches LOS bei Wind nicht genügt	27
9.3 Umgang mit beweglichen Bojen	29
9.4 Geschwindigkeit ist eine Regelgröße	29
10. Kollisionsvermeidung	30
10.1 Verfahrensvergleich	30
10.2 Das Denkwerkzeug: Velocity Obstacles	30
10.3 Die Empfehlung: szenariobasierter MPC	31
10.4 Sicherheitsabstände bei Seegang	33
11. Regelung und Echtzeitschicht	34
11.1 Warum eine Kaskade	34
11.2 Auslegung der einzelnen Schleifen	34
11.3 Anti-Windup, Ratenbegrenzung, stoßfreie Umschaltung	35
11.4 Schubsättigung: die Strategie entscheidet	35
11.5 Umsetzung auf dem Mikrocontroller	36
11.6 Zusätzliche Maßnahmen für den Seegang	37
11.7 Wann sich der Umstieg auf NMPC lohnt	37
12. Welche Rolle die KI wirklich spielt	38
12.1 Wahrnehmung: hier gewinnt das Netz	38
12.2 Regelung: hier gewinnt die Mathematik	38
12.3 Der sinnvolle Mittelweg	38
13. Gesamtverhalten in der Simulation	40
14. Simulation und Erprobung	41
14.1 Zur Simulationsumgebung	41
14.2 Was auf jeder Stufe wirklich geprüft wird	42
14.3 Aufzeichnung ist keine Kür	42
15. Sicherheit und Rückfallverhalten	43
15.1 Organisatorische Maßnahmen	43
16. Abstimmung in der richtigen Reihenfolge	44
16.1 Werkzeuge, die sich früh lohnen	44
17. Ablaufplan und Reihenfolge	45
17.1 Was parallel laufen kann und was nicht	45
17.2 Die vier größten Zeitrissen	45
18. Häufige Fehler und wie man sie vermeidet	46

Anhang A Referenzablauf als Pseudocode	47
Anhang B Parameter-Startwerte	48
Anhang C Checkliste vor jedem Einsatz	49
Vor dem Wassern	49
Im Wasser, vor dem Freigeben der Autonomie	49
Nach der Fahrt	49
Anhang D Verwendete Quellen und Literatur	50

1. Kurzfassung und Empfehlung

Die Aufgabe lässt sich mit bewährter Regelungstechnik lösen. Der entscheidende Fehler wäre, sie als ein einziges KI-Problem zu behandeln und ein neuronales Netz vom Kamerabild direkt auf die Motorbefehle zu trainieren. Der Stand der Technik bei unbemannten Wasserfahrzeugen ist eine Schichtenarchitektur, in der jede Ebene eine klar umrissene Aufgabe hat, mit eigener Taktrate läuft und einzeln getestet werden kann. Künstliche Intelligenz kommt dort zum Einsatz, wo sie klar überlegen ist: beim Erkennen der Bojen im Kamerabild.

1.1 Die sechs Kernentscheidungen

Thema	Empfehlung	Begründung in Kürze
Architektur	Fünf Schichten: Mission, Kollisionsvermeidung, Kursführung, Regelung, Schubaufteilung	jede Ebene einzeln testbar; Fehler bleiben lokal; Failsafes greifen auf jeder Ebene
Software	eigener ROS-2-Stack, harte Echtzeit ausgelagert auf einen Mikrocontroller mit micro-ROS	volle Kontrolle über jeden Knoten; Linux allein kann die 50-Hz-Schleife nicht sicher garantieren
Kursführung	ILOS bzw. ALOS statt einfachem Line-of-Sight	einfaches LOS lässt bei Querversatz durch Wind oder Strömung eine bleibende Ablage stehen – in der Simulation 1,11 m
Ausweichen	Szenariobasierter MPC (SB-MPC)	endlich viele Verhaltenskandidaten werden vorausberechnet und bewertet; nachvollziehbar, echtzeitfähig, in Seerprobungen bewährt
Kurswinkel	GNSS mit zwei Antennen (Moving Baseline), kein Magnetkompass	Motorströme und Wellenneigung verfälschen den Magnetkompass genau dann, wenn er gebraucht wird
Seegang	Wellenfilterung in der Zustandsschätzung, langsame äußere Regelschleife	gegen die Wellenbewegung zu regeln kostet Energie und Standzeit, ohne die Bahn zu verbessern

Der Satz, der dieses Dokument zusammenfasst

Das Boot fährt nicht Wegpunkte ab, sondern folgt einem Pfad, der in jedem Regelzyklus aus den aktuell gemessenen Bojenpositionen neu berechnet wird – und eine darüber liegende Schicht darf diesen Pfad jederzeit zugunsten der Kollisionsfreiheit verlassen.

1.2 Was am Ende dabei herauskommt

Abbildung 19 in Kapitel 13 zeigt das Gesamtverhalten in einer Simulation mit allen beschriebenen Bausteinen: fünf driftende Bojentore, ein querendes bewegliches Hindernis, 0,24 m/s Wind- und Strömungsversatz. Das Boot passiert alle fünf Tore in 70 s, hält dabei einen Mindestabstand von 6,2 m zum Hindernis ein und verlangsamt selbsttätig in den Kurven und im Ausweichmanöver.

1.3 Realistische Erwartungshaltung

Der zeitliche Schwerpunkt liegt nicht bei den Algorithmen, sondern bei Integration und Erprobung. Erfahrungsberichte aus studentischen Wettbewerben wie RoboBoat weisen durchgängig in dieselbe Richtung: Aufgaben, die in der Simulation zuverlässig funktionieren, scheitern auf dem Wasser an Maßstabeffekten, Rechenzeiten und Sensorrauschen. Wer den Stack selbst baut, muss zusätzlich Zeit für Infrastruktur einplanen: Nachrichtendefinitionen, Zeitstempel, Koordinatenbäume, Startskripte, Aufzeichnung. Planen Sie mindestens die Hälfte der Projektzeit für Erprobung und Nachtuning ein (Kapitel 17).

2. Aufgabenstellung und Randbedingungen

2.1 Was das System leisten muss

Aus der Aufgabenbeschreibung ergeben sich vier Teilaufgaben, die gleichzeitig laufen müssen und sich gegenseitig beeinflussen:

- **Bojen finden und verfolgen.** Die Bojen bewegen sich langsam und veränderlich. Ihre Positionen sind also Messgrößen mit Unsicherheit, keine festen Koordinaten.
- **Einen Pfad daraus bilden.** Aus Bojenpaaren entstehen Tore, aus deren Mittelpunkten ein befahrbarer Pfad. Dieser Pfad ändert sich, solange die Bojen driften.
- **Dem Pfad folgen.** Trotz Wind, Wellen und Strömung, die das Boot seitlich versetzen und den Kurs stören.
- **Hindernissen ausweichen.** Beweglichen Hindernissen, deren künftige Bahn nur geschätzt werden kann – und danach zum Pfad zurückkehren, ohne das nächste Tor zu verpassen.

Die letzten beiden Teilaufgaben stehen im Konflikt. Eine Architektur, die diesen Konflikt nicht explizit auflöst, wird entweder Tore verpassen oder Hindernisse rammen. Die Auflösung erfolgt hier über eine klare Rangordnung: Kollisionsfreiheit schlägt Pfadtreue, Pfadtreue schlägt Geschwindigkeit.

2.2 Die Umgebung ist der eigentliche Gegner

Ein Boot von einem Meter Länge ist im offenen Wasser kein verkleinertes Schiff, sondern ein qualitativ anderes Fahrzeug. Das liegt am Verhältnis von Bootslänge zu Wellenlänge.

Seegang: das Boot ist deutlich kürzer als die Wellen

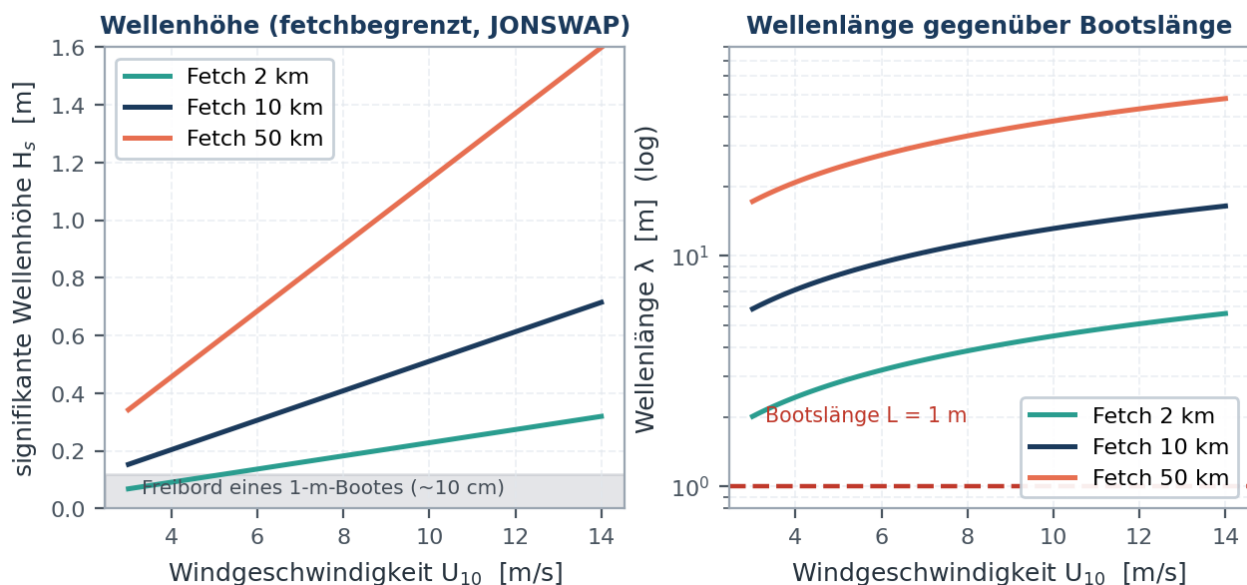


Abbildung 1: Signifikante Wellenhöhe und Wellenlänge in Abhängigkeit von Windgeschwindigkeit und Fetch (fetchbegrenzte JONSWAP-Näherung). Rechts logarithmische Achse.

Schon bei 8 m/s Wind (Windstärke 4) und 10 km Anlaufstrecke liegt die Wellenlänge bei rund 10 m, also beim Zehnfachen der Bootslänge, bei einer signifikanten Wellenhöhe um 0,45 m. Daraus folgen drei Dinge unmittelbar:

1. **Das Boot folgt der Wellenkontur.** Es durchschneidet die Wellen nicht, sondern neigt sich mit der lokalen Wasseroberfläche. Roll- und Nickwinkel von 15° bis 25° sind normal. Jede Messung, die von der Lage abhängt – Magnetkompass, Kamerageometrie, Entfernung über die Wasserlinie – wird

dadurch periodisch verfälscht.

2. **Der gemessene Kurs schwingt mit Wellenfrequenz.** Typisch sind 0,3 bis 0,7 Hz. Wenn die Regelung dagegen arbeitet, verheizt sie Akkukapazität und Motorlebensdauer, ohne die Bahn zu verbessern. Das ist der Kern der Wellenfilterung in Kapitel 7.
3. **Die Wellenhöhe kann den Freibord überschreiten.** Ein 1-m-Boot hat typisch 8 bis 15 cm Freibord. Bei einer signifikanten Wellenhöhe von 0,45 m kommt regelmäßig Wasser über. Wasserdichtigkeit ist deshalb keine Nebensache, sondern Voraussetzung für alles andere.

Auslegungsfenster festlegen, bevor gebaut wird

Legen Sie schriftlich fest, bis zu welcher Windstärke das System arbeiten soll, und leiten Sie daraus Wellenhöhe, benötigten Schub und Freibord ab. Ein System, das für Bft 3 ausgelegt ist und bei Bft 5 eingesetzt wird, versagt nicht graduell, sondern vollständig. Für diesen Entwurf ist Bft 4 (bis etwa 8 m/s Wind, Wellenhöhe bis 0,5 m bei küstennahem Fetch) als Obergrenze angesetzt.

2.3 Physikalische Grenzen des Rumpfes

Ein Verdrängerrumpf erreicht seine Rumpfgeschwindigkeit bei einer Froude-Zahl von etwa 0,4. Mit der Wasserlinienlänge $L = 1,0$ m folgt:

$$v_{\text{hull}} = 0,4 \sqrt{gL} = 0,4 \sqrt{9,81 \cdot 1,0} \text{ m/s} \approx 1,25 \text{ m/s}$$

Oberhalb dieses Wertes steigt der Widerstand stark an, weil das Boot gegen sein eigenes Bugwellensystem anfährt. Mehr Motorleistung bringt dann kaum noch Fahrt, wohl aber Stromverbrauch. Für die Auslegung heißt das: 1,5 m/s Marschfahrt und 2,0 m/s Höchstfahrt sind sinnvolle Zielwerte; alles darüber erfordert einen Gleit- oder Katamaranrumpf. Praktisch relevant ist ohnehin nicht die Höchstfahrt, sondern die *Schubreserve*: Das Boot muss auch dann noch steuerbar bleiben, wenn Wind und Wellenwiderstand den Bedarf verdreifachen.

2.4 Anforderungsliste

Diese Liste ist der Bezugspunkt für alle späteren Abnahmen. F = Funktion, Q = Qualität, S = Sicherheit.

Nr.	Anforderung	Zielwert / Nachweis
F1	Bojentore in vorgegebener Reihenfolge durchfahren	alle Tore einer Runde, Nachweis in Stufe 4 und 5
F2	Bewegliche Bojen bis 0,3 m/s Drift verfolgen	Trackverlust unter 5 % der Zeit
F3	Beweglichen Hindernissen ausweichen	Mindestabstand 3 m in ruhigem, 5 m in bewegtem Wasser
F4	Rückkehr auf den Pfad nach Ausweichmanöver	nächstes Tor wird trotzdem passiert
Q1	Bahnabweichung bei Marschfahrt in ruhigem Wasser	Querablage unter 0,5 m im Mittel
Q2	Bahnabweichung bei Bft 4	Querablage unter 1,5 m im Mittel, keine bleibende Ablage
Q3	Einsatzdauer	mindestens 45 min bei realistischem Lastprofil
Q4	Zeitverhalten der Regelschleife	50 Hz auf dem Mikrocontroller, Jitter unter 1 ms
Q5	Rechenlast auf dem Linux-Rechner	unter 60 % Dauerlast, keine verworfenen Kamerabilder
S1	Funk-Not-Aus, unabhängig vom Autonomiepfad	Motoren stromlos in unter 0,5 s, in Stufe 3 nachgewiesen
S2	Rückfallverhalten bei Ausfall von GNSS, Kamera oder Rechner	definierter sicherer Zustand, siehe Kapitel 15
S3	Geofence um das Einsatzgebiet	automatisches Stoppen oder Rückkehr bei Verletzung

3. Systemarchitektur

Die Architektur folgt dem in der Schiffsautomatisierung üblichen GNC-Modell: **Guidance** beantwortet die Frage „wohin“, **Navigation** die Frage „wo bin ich und wie bewege ich mich“, **Control** die Frage „welche Kräfte sind dafür nötig“. Darüber liegen Missionslogik und Kollisionsvermeidung, darunter die Schubaufteilung auf die beiden Motoren.

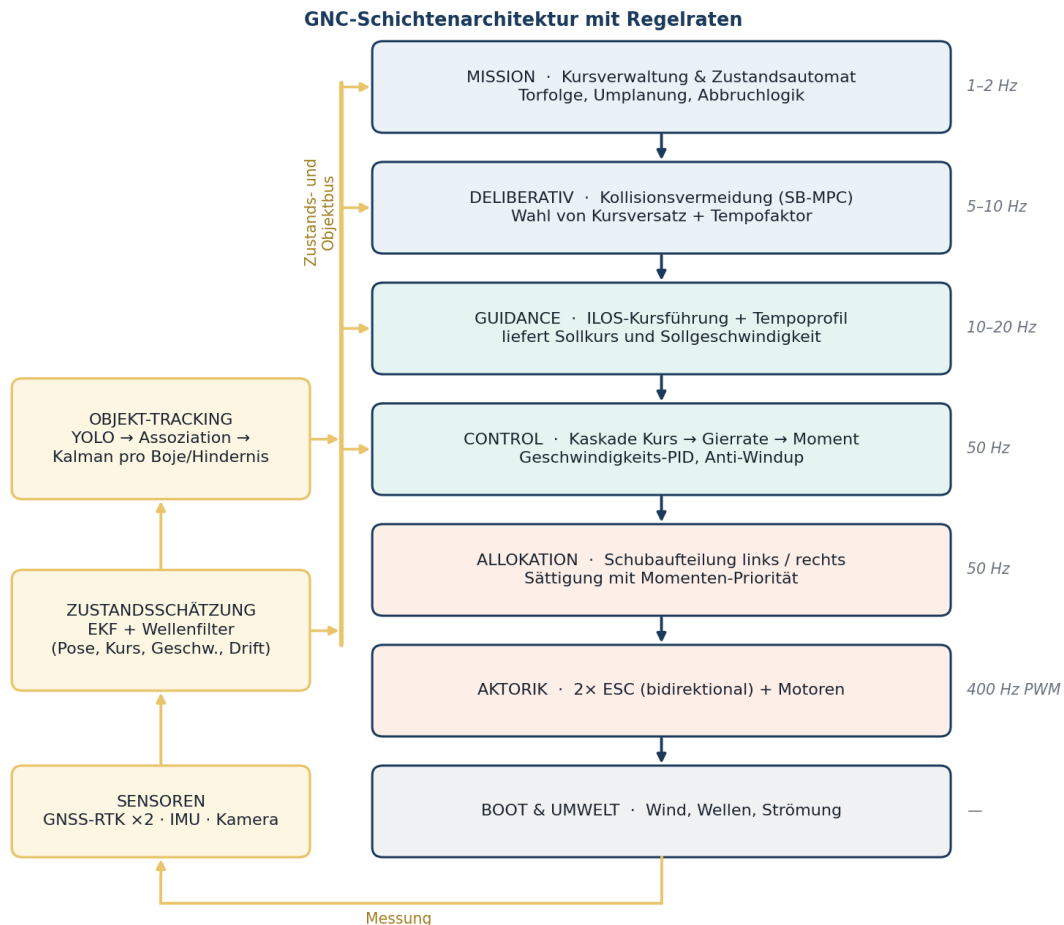


Abbildung 2: Schichtenarchitektur mit Taktraten. Die gelbe Kette links liefert Zustand und Objektliste, die blaue Kette rechts erzeugt daraus die Stellgrößen.

3.1 Warum Schichten und nicht ein einziges Netz

Ein Ende-zu-Ende gelerntes System erscheint verlockend, weil es die Modellbildung erspart. Es hat für diese Aufgabe aber drei Nachteile, die in der Praxis entscheidend sind:

- **Nicht debuggbar.** Fährt das Boot am Tor vorbei, lässt sich nicht feststellen, ob die Boje falsch erkannt, falsch verortet, falsch zugeordnet oder nur falsch angesteuert wurde. In einer Schichtenlösung zeigt ein Blick auf die Topics, welche Ebene versagt hat.
- **Nicht absicherbar.** Für PID- und ILOS-Regelung existieren Stabilitätsbeweise. Für eine gelernte Politik existiert bestenfalls eine Teststatistik – und Sie können sie nicht für alle Seegänge testen.
- **Datenhunger.** Die gelernte Politik müsste alle Kombinationen aus Seegang, Bojenkonstellation und Hindernisbewegung gesehen haben. Die Übertragung aus der Simulation auf das reale Wasser ist bei stark gestörten Systemen der schwierigste Teil.

Kapitel 12 zeigt, wie sich lernende Verfahren trotzdem sinnvoll einbinden lassen – als begrenzter Zusatzanteil auf einem geprüften Basisregler, nicht als Ersatz.

3.2 Takt- und Latenzbudget

Die Raten sind kein Detail, sondern eine Entwurfsentscheidung. Eine Kaskade funktioniert nur, wenn die innere Schleife deutlich schneller ist als die äußere – Faustregel: Faktor fünf.

Ebene	Rate	Läuft auf	Zulässige Latenz
Missionslogik / Zustandsautomat	1–2 Hz	Linux	500 ms
Kollisionsvermeidung SB-MPC	5–10 Hz	Linux	150 ms
Objekterkennung und Tracking	10–15 Hz	Linux (GPU)	150 ms
Kursführung ILOS	10–20 Hz	Linux	80 ms
Zustandsschätzung EKF	50 Hz	Linux	40 ms
Kurs- und Gierratenregelung	50 Hz	Mikrocontroller	20 ms
Schubaufteilung und PWM	50 / 400 Hz	Mikrocontroller	10 ms

Die Aufteilung auf zwei Rechner ist bewusst gewählt und wird in Kapitel 4 begründet: Alles, was sicherheitsrelevant und zeitkritisch ist, läuft auf einem Mikrocontroller. Alles, was rechenintensiv und tolerant gegen einzelne Aussetzer ist, läuft auf dem Linux-Rechner. Fällt der Linux-Rechner aus, hält der Mikrocontroller Kurs und löst nach einer definierten Zeit den Rückfallmodus aus – das Boot wird nicht steuerlos.

4. Softwarearchitektur mit ROS 2

Ein eigener ROS-2-Stack gibt Ihnen die volle Kontrolle über jeden Baustein und ist die richtige Wahl, wenn Sie Verfahren wie ILOS und SB-MPC selbst implementieren wollen. Der Preis ist, dass Sie alles selbst bauen müssen, was ein fertiger Autopilot mitbringt: Zustandsschätzung, Regelung, Rückfallverhalten, Zeitverhalten. Dieses Kapitel beschreibt, wie sich das strukturieren lässt, ohne in die typischen Fallen zu laufen.

4.1 Die Grundentscheidung: Linux allein genügt nicht

ROS 2 läuft auf einem gewöhnlichen Linux. Dessen Standardkern ist auf Durchsatz optimiert, nicht auf garantierte Reaktionszeiten. Das Projekt *ros2_control* beschreibt genau diesen Punkt: Für möglichst geringe Laufzeitschwankung soll der Regelungsprozess mit der Echtzeit-Ablaufplanung SCHED_FIFO auf hoher Priorität laufen, der Arbeitsspeicher gesperrt werden und ein Echtzeit- oder zumindest Low-Latency-Kern zum Einsatz kommen. Selbst mit all diesen Maßnahmen sind auf einem Einplatinenrechner einzelne Ausreißer im Millisekundenbereich nicht ausgeschlossen.

Für eine Kursregelung ist eine verpasste Abtastung meist verkräftbar. Für die Sicherheitsabschaltung und die Erzeugung der Motorsignale ist sie es nicht. Deshalb die Empfehlung: **harte Echtzeit auf einen Mikrocontroller auslagern** und ihn über micro-ROS als vollwertigen ROS-2-Teilnehmer einbinden.

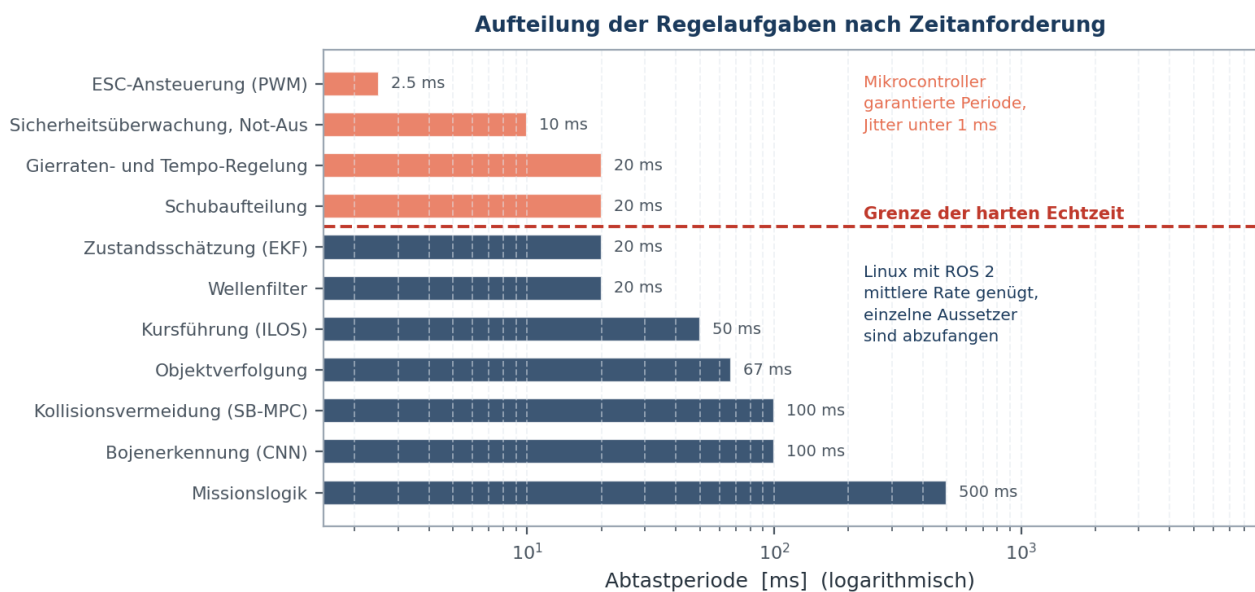


Abbildung 3: Einordnung der Aufgaben nach Abtastperiode. Alles unterhalb der roten Linie hat harte Zeitanforderungen und gehört auf den Mikrocontroller.

4.2 micro-ROS als Brücke

micro-ROS bringt die ROS-2-Konzepte – Knoten, Topics, Dienste, Parameter – auf Mikrocontroller. Es besteht aus zwei Teilen: einem schlanken Client auf dem Mikrocontroller und einem Agenten auf dem Linux-Rechner, der als Brücke in das DDS-Netz wirkt. Die Verbindung läuft über das Protokoll DDS-XRCE, das für ressourcenarme Geräte entworfen wurde, typischerweise über eine serielle Schnittstelle. Unterstützt werden gängige 32-Bit-Mikrocontroller mit wenigen Dutzend Kilobyte Arbeitsspeicher, wahlweise mit Echtzeitbetriebssystem oder ohne.

- **Praktischer Vorteil.** Der Mikrocontroller erscheint in der Topic-Liste wie jeder andere Knoten. Aufzeichnung, Diagnose und Visualisierung funktionieren ohne Sonderbehandlung.

- **Wichtiger Unterschied.** Der Executor auf der Mikrocontroller-Seite arbeitet die Rückrufe in fester, vom Entwickler vorgegebener Reihenfolge ab. Das ist weniger flexibel als auf dem Linux-Rechner, aber genau das, was für vorhersagbares Zeitverhalten gebraucht wird.
- **Kandidaten für die Hardware.** Teensy 4.1 (sehr einfach in Betrieb zu nehmen), STM32 (mehr Peripherie, mehr Einarbeitung), ESP32 (günstig, aber Funkbetrieb und harte Echtzeit vertragen sich schlecht).

4.3 Der Knotengraph

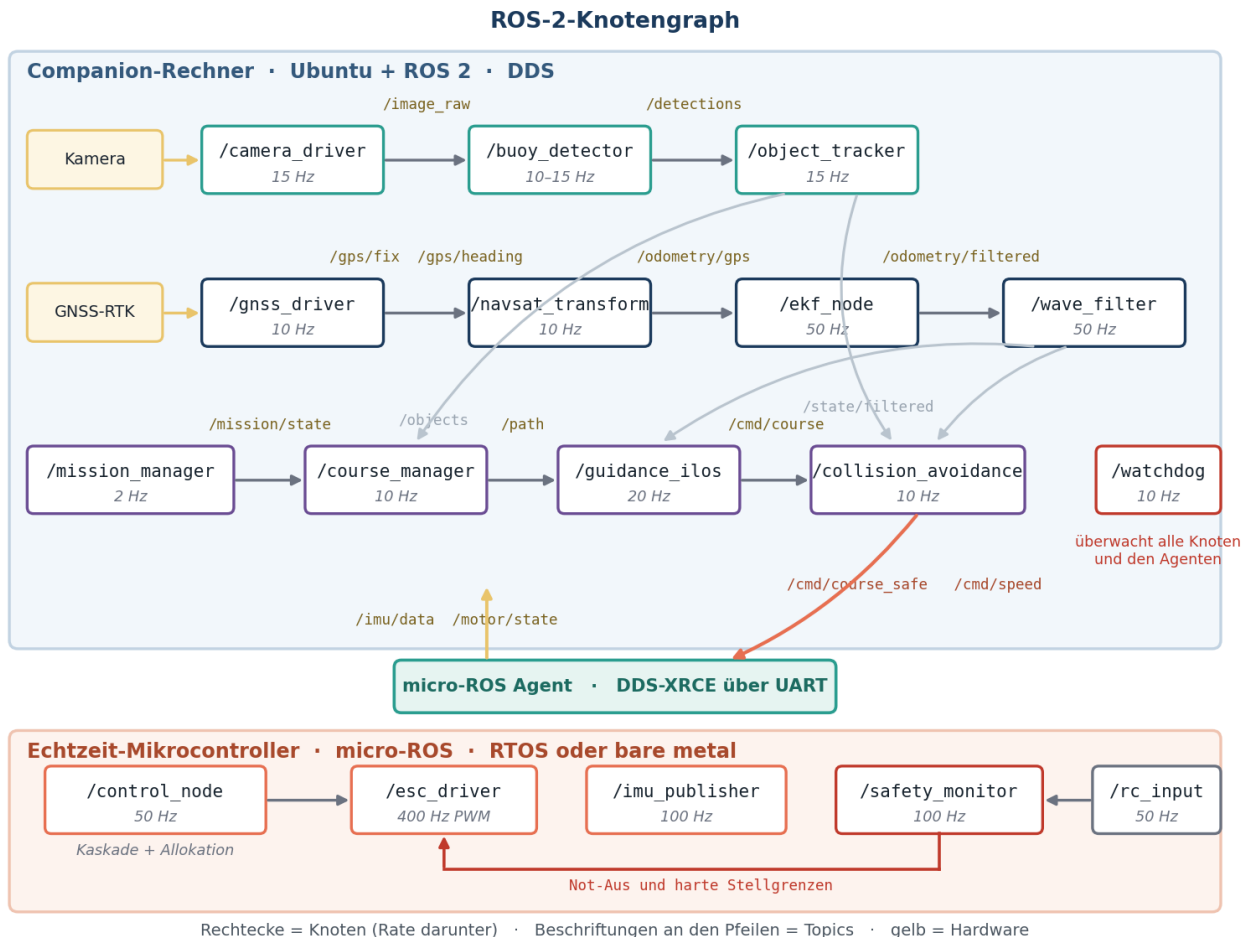


Abbildung 4: Aufteilung der Knoten auf Linux-Rechner und Mikrocontroller. Die Beschriftungen an den Pfeilen sind die Topics, die Angaben unter den Knotennamen deren Taktrate.

Ein Knoten hat genau eine Aufgabe. Diese Regel klingt banal, ist aber der Unterschied zwischen einem Stack, den man in einem halben Jahr noch versteht, und einem, den man neu schreibt. Die wichtigsten Schnittstellen:

Topic	Nachrichtentyp	Herausgeber	Inhalt
/gps/fix	sensor_msgs/NavSatFix	/gnss_driver	Position mit Fixgüte
/gps/heading	sensor_msgs/Imu	/gnss_driver	wahrer Kurs aus der Doppelantenne
/imu/data	sensor_msgs/Imu	/imu_publisher	Drehraten und Beschleunigungen
/odometry/filtered	nav_msgs/Odometry	/ekf_node	geschätzte Pose und Geschwindigkeit
/state/filtered	nav_msgs/Odometry	/wave_filter	wellengefilterter Zustand für die Führung
/detections	vision_msgs/Detection2DArray	/buoy_detector	Bildkoordinaten und Klasse

Topic	Nachrichtentyp	Herausgeber	Inhalt
/objects	eigener Typ	/object_tracker	Objektliste mit Position, Drift, Kovarianz, Trackalter
/path	nav_msgs/Path	/course_manager	aktueller Sollpfad aus den Torzentren
/cmd/course	eigener Typ	/guidance_ilos	Sollkurs und Sollgeschwindigkeit
/cmd/course_safe	eigener Typ	/collision_avoidance	freigegebener Sollkurs nach Prüfung
/motor/state	eigener Typ	/esc_driver	gestellte Schübe, Ströme, Sättigungsanzeige

Eigene Nachrichtentypen sparsam einsetzen

Nutzen Sie Standardtypen, wo es geht – dann funktionieren fertige Werkzeuge zur Visualisierung und Auswertung ohne Zusatzarbeit. Eigene Typen lohnen sich dort, wo der Standard nicht passt, vor allem für die Objektliste und die Fahrbefehle. Jede eigene Nachricht braucht einen Zeitstempel im Kopf und eine Angabe des Bezugssystems.

4.4 Sechs Dinge, die man beim eigenen Stack falsch machen kann

Thema	Falle	Richtige Umsetzung
Zeitstempel	Empfangszeit statt Messzeit eintragen	jeder Treiber stempelt mit der Messzeit; der EKF verlässt sich darauf. Eine einheitliche Zeitbasis über alle Knoten ist Pflicht.
Koordinatenbaum	Bezugssysteme nur in Kommentaren führen	sauberen Transformationsbaum aufbauen (Karte → Odometrie → Bootsumpf → Sensoren) und die festen Anteile aus einer Beschreibungsdatei veröffentlichen
Dienstgüte (QoS)	überall die Voreinstellung lassen	Sensordaten mit kurzer Warteschlange und „best effort“, Befehle und Zustandswechsel „reliable“. Sonst sammeln sich alte Kamerabilder an und die Regelung arbeitet auf veralteten Daten.
Ausführung	jeder Knoten ein eigener Prozess	zeitkritische Knoten in einem Prozess zusammenfassen und die Rückrufe auf getrennte Gruppen legen; das spart Kopiervorgänge und Kontextwechsel
Zustandswechsel	Betriebsart über globale Variablen	Lebenszyklus-Knoten verwenden: Knoten haben definierte Zustände, und der Wechsel nach „aktiv“ ist ein nachvollziehbares Ereignis
Aufzeichnung	erst kurz vor dem Seeversuch einbauen	von Anfang an mitschreiben. Die Aufzeichnung ist gleichzeitig Fehlersuche, Regressionstest und Trainingsdatenquelle.

4.5 Was Sie nicht selbst schreiben sollten

Ein eigener Stack heißt nicht, alles selbst zu programmieren. Für drei Bausteine gibt es erprobte Pakete, deren Nachbau reine Zeitverschwendung wäre:

- **Zustandsschätzung.** Das Paket *robot_localization* bietet einen EKF und einen UKF, die beliebig viele Sensorquellen fusionieren, sowie einen Knoten, der GNSS-Koordinaten in das kartesische Weltsystem überführt. Der übliche Aufbau verwendet zwei Filterinstanzen: eine lokale, stetige Schätzung ohne GNSS und eine globale Schätzung mit GNSS.
- **Kameraanbindung und Kalibrierung.** Die vorhandenen Treiber- und Kalibrierpakete sind erprobt und sparen Wochen.
- **Simulation.** Eine Wassenumgebung mit Wind- und Wellenmodell selbst zu bauen ist ein eigenes Projekt (Kapitel 14).

Selbst schreiben sollten Sie dagegen alles, was die eigentliche Aufgabe ausmacht: Wellenfilter, Torbildung, ILOS-Kursführung, SB-MPC, Kaskadenregler, Schubaufteilung und die Sicherheitslogik. Das sind zusammen einige Hundert Zeilen – überschaubar, wenn die Struktur stimmt.

5. Bootsmodell und Aktorik

Ein einfaches Modell ist kein akademischer Selbstzweck. Es wird an drei Stellen konkret gebraucht: für die Reglerauslegung, für die Vorausberechnung im SB-MPC und für die Simulation der Teststufen 0 und 1.

5.1 Bezugssysteme und Zustandsgrößen

Koordinatensysteme und Aktorik

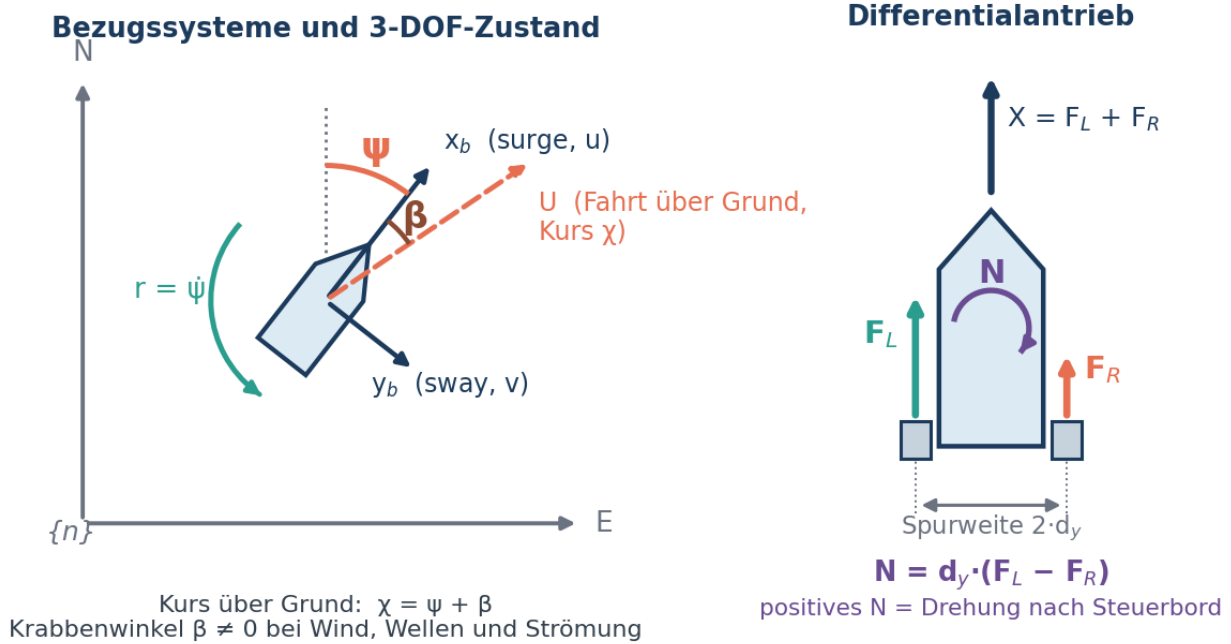


Abbildung 5: Links das ortsfeste NED-System und das körperfeste System mit den drei Freiheitsgraden der Horizontalbewegung. Rechts die Wirkung des Differentialantriebs.

Für die Horizontalbewegung genügen drei Freiheitsgrade: Längsfahrt (surge, u), Querversatz (sway, v) und Gieren (yaw, r). Roll, Nick und Hub werden nicht geregelt, sind aber als Störgrößen und als Fehlerquelle für die Sensorik vorhanden. Der Zustandsvektor lautet:

$$\eta = \begin{pmatrix} x \\ y \\ \psi \end{pmatrix}, \quad \nu = \begin{pmatrix} u \\ v \\ r \end{pmatrix}, \quad \dot{\eta} = \mathbf{R}(\psi) \nu$$

mit der Drehmatrix

$$\mathbf{R}(\psi) = \begin{pmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Zentral für das Verständnis ist die Unterscheidung von **Kurswinkel** ψ (wohin der Bug zeigt) und **Kurs über Grund** χ (wohin sich das Boot tatsächlich bewegt). Die Differenz ist der Krabbenwinkel:

$$\beta = \chi - \psi = \arctan\left(\frac{v}{u}\right)$$

Bei Seitenwind oder Querströmung ist β ungleich null, und genau daran scheitert eine naive Kursregelung: Das Boot hält den Bug exakt auf dem Sollwert und versetzt trotzdem seitwärts. Kapitel 9 behandelt die Kompensation.

5.2 Das Nomoto-Modell für die Drehbewegung

Für die Reglerauslegung reicht ein Modell erster Ordnung zwischen Stellgröße und Gierrate, das sogenannte Nomoto-Modell:

$$T \dot{r} + r = K \delta$$

r	Gierrate in rad/s
δ	normierte Stellgröße, dimensionslos im Bereich -1 bis $+1$
K	Verstärkung in $1/s$ – wie kräftig das Boot dreht
T	Zeitkonstante in s – wie träge es dabei ist

Für ein 1-m-Boot mit zwei Motoren liegen typische Werte bei $K = 0,8 \dots 1,5 \text{ s}^{-1}$ und $T = 0,8 \dots 1,5 \text{ s}$ – das Boot ist also sehr wendig. Diese Wendigkeit ist Segen und Fluch: Enge Tore sind kein Problem, aber die Gefahr des Aufschwingens im Seegang ist hoch.

5.3 Schubaufteilung

Aus den beiden Motorschüben ergeben sich Längskraft und Giermoment. Mit dem halben Motorabstand d_y gilt:

$$\begin{aligned} X &= F_L + F_R & N &= d_y (F_L - F_R) \\ F_L &= \frac{X}{2} + \frac{N}{2d_y} & F_R &= \frac{X}{2} - \frac{N}{2d_y} \end{aligned}$$

Wichtig ist die Vorzeichenkonvention: Ein positives Giermoment N dreht den Bug nach Steuerbord, wenn der *backbordseitige* Motor stärker schiebt. Ein Vorzeichenfehler an dieser Stelle führt zu einem System, das exakt in die falsche Richtung ausweicht – und das fällt in der Simulation oft erst spät auf. Prüfen Sie es im Trockentest (Stufe 3) von Hand nach.

5.4 Systemidentifikation am realen Boot

K und T werden nicht geschätzt, sondern gemessen. Das Standardverfahren ist der Zickzack-Versuch: Die Stellgröße wird auf einen festen Wert gelegt, bis der Kurs 20° überschritten hat, dann umgeschaltet, und so weiter. Aus Stellgröße und Gierrate lassen sich die Parameter per Ausgleichsrechnung bestimmen – das Nomoto-Modell ist dafür linear in den Unbekannten:

$$\underbrace{\begin{pmatrix} -\dot{r}_1 & \delta_1 \\ \vdots & \vdots \\ -\dot{r}_n & \delta_n \end{pmatrix}}_{\mathbf{A}} \begin{pmatrix} T \\ K \end{pmatrix} = \begin{pmatrix} r_1 \\ \vdots \\ r_n \end{pmatrix} \implies \hat{\boldsymbol{\theta}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{r}$$

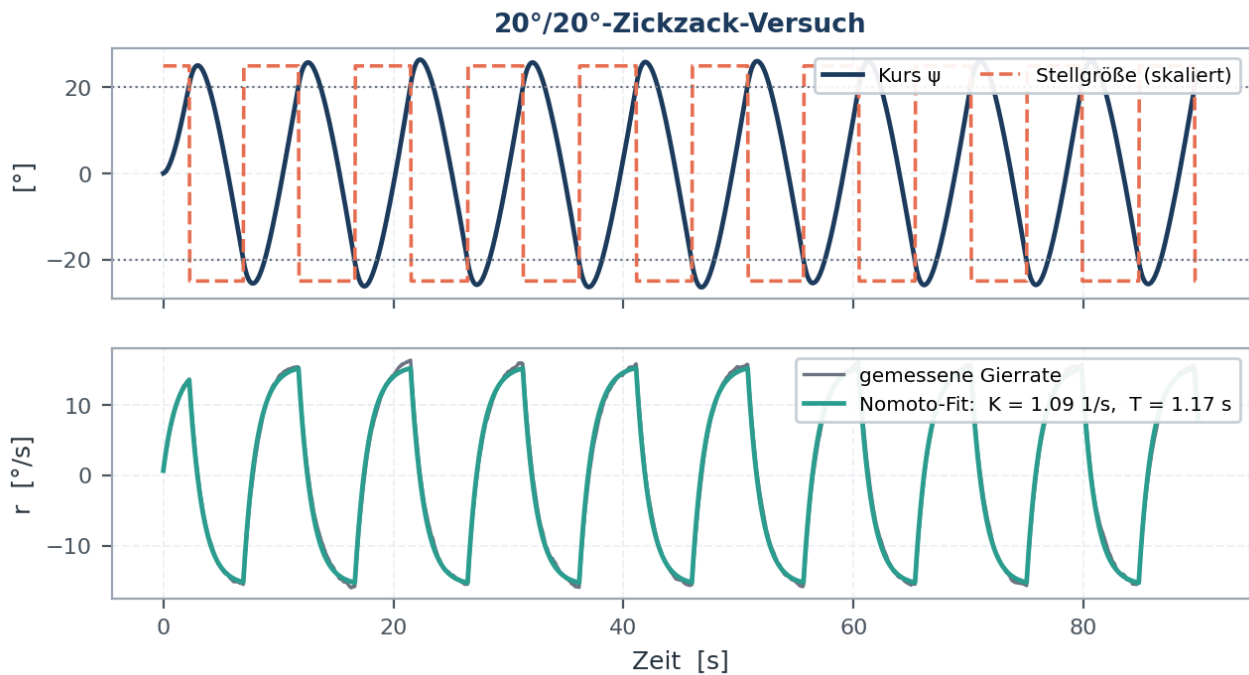
Systemidentifikation: aus dem Zickzack-Versuch folgen K und T 

Abbildung 6: Simulierter 20°/20°-Zickzack-Versuch. Aus den Messdaten wurden $\hat{K} = 1,09 \text{ s}^{-1}$ und $\hat{T} = 1,17 \text{ s}$ geschätzt; die zugrunde liegenden wahren Werte waren 1,15 und 1,30. Die verbleibende Abweichung stammt aus dem Rauschen der numerisch differenzierten Gierrate.

Praktische Hinweise: Fahren Sie den Versuch bei konstanter Marschgeschwindigkeit und in möglichst ruhigem Wasser. Wiederholen Sie ihn für beide Drehrichtungen – ein asymmetrisches Ergebnis deutet auf unterschiedliche Propeller, schiefen Schwerpunkt oder einen kalibrierfehlerhaften Fahrtregler hin. Wiederholen Sie ihn außerdem für zwei Geschwindigkeiten: K und T hängen von der Fahrt ab, und die Regelung sollte für die langsamere Variante ausgelegt werden, weil dort die Steuerwirkung am schwächsten ist.

6. Hardware: Auslegung und Auswahl

6.1 Wie viel Schub wird tatsächlich gebraucht?

Die folgende Überschlagsrechnung geht von einem Boot mit 6 kg Verdrängung, 1 m Länge und etwa 0,20 m² benetzter Fläche aus. Setzen Sie Ihre eigenen Werte ein – die Rechnung ändert sich nicht, nur die Zahlen. Der Reibungsbeiwert folgt der ITTC-Korrelation:

$$\text{Re} = \frac{v L}{\nu} = \frac{1,5 \cdot 1,0}{1,19 \cdot 10^{-6}} \approx 1,26 \cdot 10^6$$

$$C_f = \frac{0,075}{(\log_{10} \text{Re} - 2)^2} \approx 0,0045 \quad \Rightarrow \quad C_t \approx 0,010$$

Damit ergibt sich der Glattwasserwiderstand zu

$$R = \frac{1}{2} \rho S C_t v^2 = \frac{1}{2} \cdot 1025 \cdot 0,20 \cdot 0,010 \cdot 1,5^2 \text{ N} \approx 2,3 \text{ N}$$

und bei 2,0 m/s rund 4,5 N. Hinzu kommen zwei Zuschläge:

Anteil	Ansatz	Betrag
Glattwasserwiderstand bei 2,0 m/s	siehe Rechnung oben	4,5 N
Zuschlag Wellenwiderstand im Seegang	Faktor 2 bis 2,5	rund 11 N
Windwiderstand	$A = 0,03 \text{ m}^2$, $c_d = 0,8$, 12 m/s scheinbarer Wind	2,1 N
Auslegungsbedarf gesamt		13 bis 15 N

Das sind etwa 1,5 kgf – erstaunlich wenig. Der Schub wird jedoch nicht für die Fahrt gebraucht, sondern für die **Steuerautorität**: Das Boot muss sich auch gegen eine anlaufende Welle in den Kurs zwingen können, und es braucht Rückwärtsschub für enge Manöver. Als Faustregel sollte der maximale Standschub etwa das Fünf- bis Achtfache des Fahrtwiderstands betragen.

6.2 Motoren und Fahrtregler

Die naheliegende Wahl ist der Blue-Robotics-T200. Er ist für genau diesen Anwendungsfall gebaut, vollständig geflutet und damit wassergekühlt, aus seewasserfesten Werkstoffen, und er ist in der ROS-Welt so weit verbreitet, dass jedes Problem bereits jemand anderes hatte. Die Herstellerangaben:

Betriebsspannung	Schub vorwärts	Schub rückwärts	Strom bei Vollgas	Leistung
12 V (3S)	3,71 kgf	2,92 kgf	17 A	205 W
16 V (4S) – Nennbetrieb	5,25 kgf	4,10 kgf	24 A	390 W
20 V (5S) – Maximum	6,70 kgf	5,05 kgf	32 A	645 W

Zulässiger Bereich 7 bis 20 V, empfohlen 12 bis 16 V; oberhalb von 20 V erlischt die Freigabe des Herstellers. Propellerdurchmesser 76 mm, Masse 427 g in Luft. Jeder Motor benötigt einen eigenen bidirektionalen Fahrtregler; ein Fahrtregler kann nicht zwei Motoren treiben, weil er die Rotorlage sensorlos mitschätzen muss.

Rechnen Sie nach, bevor Sie bestellen

Zwei T200 bei 16 V liefern zusammen rund 103 N Standschub – etwa das Siebenfache des oben ermittelten Bedarfs. Das ist genau die gewünschte Reserve, bedeutet aber auch: 48 A Spitzenstrom. Akku, Sicherung, Kabelquerschnitt und Steckverbinder müssen dafür ausgelegt sein, auch wenn dieser Strom im Betrieb nie dauerhaft fließt. Begrenzen Sie die Stellgröße in der Software auf 60 bis 70 %; die volle Leistung bleibt als Reserve für Böen und Ausweichmanöver.

6.3 Sensorik – hier entscheidet sich der Erfolg**Position und Kurswinkel**

Die wichtigste einzelne Empfehlung dieses Dokuments betrifft die Kurswinkelmessung. Verwenden Sie **keinen Magnetkompass** als primäre Quelle. Er wird durch die Motorströme gestört, deren Betrag genau dann am größten ist, wenn viel gesteuert wird, und er wird durch die Neigung im Seegang verfälscht, weil die Horizontalkomponente des Erdfelds fehlerhaft projiziert wird.

Die Lösung ist ein GNSS-Empfänger mit zwei Antennen im Moving-Baseline-Betrieb. Er misst den Vektor zwischen den Antennen mit Trägerphasengenaugigkeit und liefert daraus den *wahren* Kurs, unabhängig von Magnetfeldern und in erster Näherung unabhängig von der Neigung. Die Winkelgenauigkeit skaliert mit der Basislänge b :

$$\sigma_\psi \approx \frac{\sigma_b}{b} \quad \text{Größenordnung: } 0,2^\circ \dots 0,4^\circ \text{ bei } b = 1 \text{ m}$$

- **Antennenabstand.** Bei einem 1-m-Boot sind 0,4 bis 0,6 m realistisch, also etwa $0,4^\circ$ bis $0,8^\circ$ Kursfehler. Montieren Sie die Antennen so weit auseinander wie irgend möglich.
- **RTK-Korrekturen.** Für Positionsgenaugigkeit im Zentimeterbereich brauchen Sie zusätzlich eine Basisstation oder einen Korrekturdienst. Für die reine Kurswinkelmessung ist das nicht nötig; die Basislinie wird zwischen den beiden eigenen Antennen aufgelöst.
- **Rückfallebene.** Halten Sie Kompass und den aus der Bewegung abgeleiteten Kurs über Grund als Rückfall bereit, aber mit geringer Gewichtung. Der Kurs über Grund ist bei Fahrt brauchbar, im Stand oder bei langsamer Fahrt jedoch wertlos.

Kamera

Für die Bojenerkennung ist eine Kamera mit **Global Shutter** vorzuziehen. Ein Rolling Shutter verzerrt das Bild, wenn sich das Boot während des Auslesens dreht – im Seegang also permanent. Ein Polarisationsfilter reduziert die Spiegelungen auf der Wasseroberfläche erheblich und ist eine der billigsten Maßnahmen mit großer Wirkung. Montieren Sie die Kamera so hoch wie stabil möglich und schützen Sie das Fenster gegen Gischt; eine hydrophobe Beschichtung hilft mehr als jeder Bildfilter.

Was Sie wahrscheinlich nicht brauchen

2D-LiDAR arbeitet auf dem Wasser schlecht: Die Spiegelung erzeugt Geisterechos, und bei Wellengang zeigt die Scanebene abwechselnd in den Himmel und ins Wasser. Für ein 1-m-Boot kommen Masse und Leistungsaufnahme hinzu. Marineradar ist wirksam, aber für diese Fahrzeuggröße zu groß. Bleiben Sie bei Kamera plus GNSS und investieren Sie die gesparte Zeit in die Erprobung.

6.4 Rechner

Aufgabe	Empfehlung	Bemerkung
Linux-Rechner	Raspberry Pi 5 oder Jetson Orin Nano	Pi 5 genügt bei kleinem Erkennungsmodell oder Auslagerung auf einen Beschleuniger; Jetson bei größerem Modell in voller Bildrate
Betriebssystem	Ubuntu mit Low-Latency- oder Echtzeitkern	Regelungsprozesse mit SCHED_FIFO und gesperrtem Speicher betreiben
Mikrocontroller	Teensy 4.1, STM32 oder vergleichbar	600 MHz Klasse, Fließkommaeinheit, mindestens vier Hardware-Timer für PWM und Zeitbasis
Kopplung	micro-ROS über UART, 921 kBd oder mehr	keine Funkstrecke im Regelpfad

6.5 Energieversorgung

Lastprofil für die Auslegung: Antrieb im Mittel 120 bis 200 W (Spitzen deutlich höher), Elektronik 20 bis 35 W. Mit einem 4S-Li-Ion-Paket (14,4 V nominal) und 10 Ah ergeben sich

$$E = U Q = 14,4 \text{ V} \cdot 10 \text{ Ah} = 144 \text{ Wh} \quad \Rightarrow \quad t = \frac{144 \text{ Wh}}{200 \text{ W}} \approx 43 \text{ min}$$

Bei 120 W Gesamtlast sind es etwa 70 min. Für die geforderten 45 min Einsatzdauer ist 10 Ah die Untergrenze; 15 bis 20 Ah geben Luft für Nachtuning ohne Akkuwechsel.

- Logik und Antrieb über getrennte Wandler versorgen. Der Spannungseinbruch beim Anlauf beider Motoren darf den Rechner nicht zurücksetzen.
- Hauptsicherung und ein mechanisch erreichbarer Not-Aus gehören in den Leistungspfad, nicht in den Signalfad.
- Strom und Spannung protokollieren. Der Energieverbrauch pro gefahrenem Meter ist die aussagekräftigste Kennzahl dafür, ob die Regelung gegen die Wellen arbeitet.

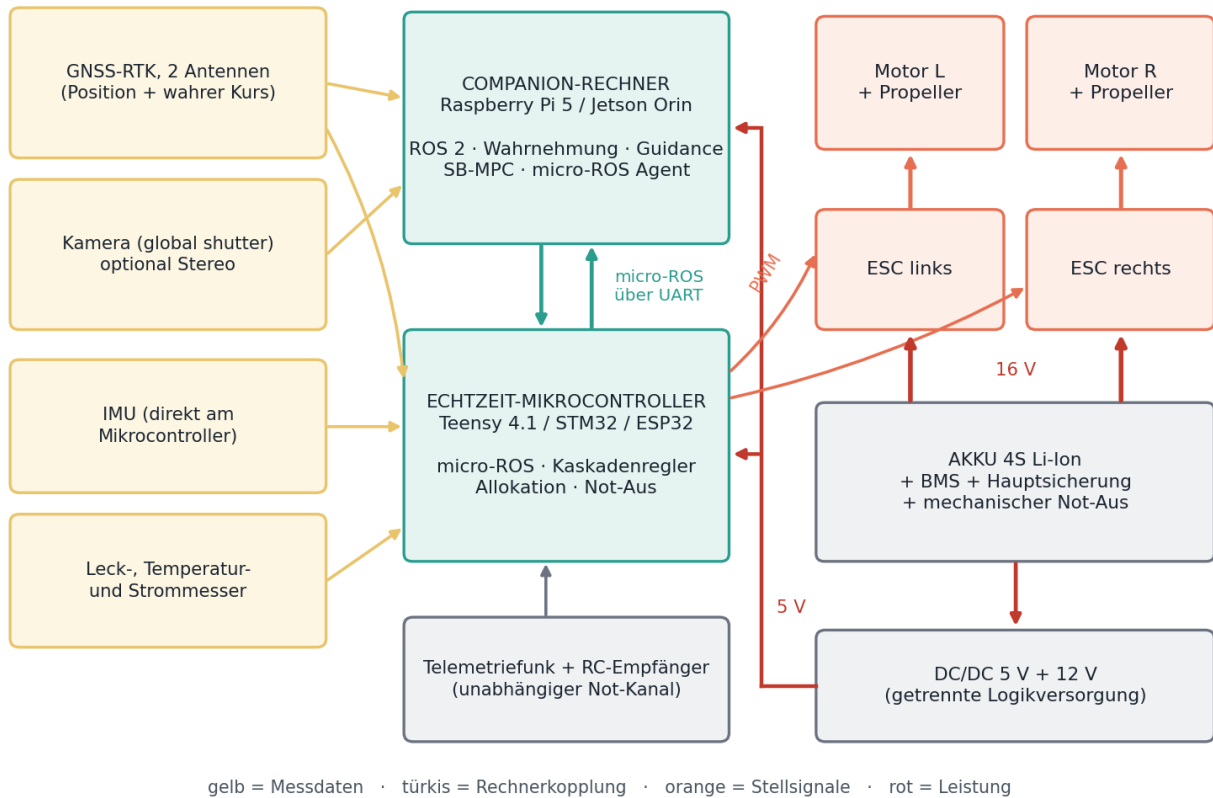
Hardware-Topologie: Linux-Rechner und Echtzeit-Mikrocontroller

Abbildung 7: Hardware-Topologie. Die getrennten Pfade für Messdaten, Rechnerkopplung, Stellsignale und Leistung sollten sich auch in der realen Verkabelung wiederfinden.

6.6 Stückliste als Ausgangspunkt

Preise als grobe Orientierung; ohne Kleinteile, Kabel und Steckverbinder.

Pos.	Komponente	Menge	Richtpreis gesamt
1	Unterwassermotor T200 oder gleichwertig	2	ca. 460 €
2	Bidirektionaler Fahrtregler	2	ca. 80 €
3	Mikrocontrollerplatine (Teensy 4.1 / STM32)	1	30-60 €
4	Trägerplatine mit Pegelwandlern und Schutzbeschaltung	1	40-120 €
5	GNSS-Empfänger mit zwei Antennen (Moving Baseline)	1	350-700 €
6	GNSS-Antennen mit Grundplatte	2	60-160 €
7	Linux-Rechner (Pi 5 / Jetson Orin Nano)	1	120-350 €
8	Kamera, global shutter, optional Stereo	1	150-450 €
9	IMU-Modul für den Mikrocontroller	1	30-90 €
10	Akku 4S Li-Ion, 10-20 Ah, mit BMS	1	120-250 €
11	DC/DC-Wandler, Sicherungen, Not-Aus	1 Satz	60-120 €
12	Telemetriefunk und Fernsteuerung mit Failsafe	1 Satz	150-300 €
13	Wasserdichtes Gehäuse, Durchführungen, Kleinteile	1 Satz	100-250 €
Summe (Richtwert)			rund 1.750 bis 3.400 €

7. Zustandsschätzung und Wellenfilterung

Die Zustandsschätzung verschmilzt GNSS, Trägheitssensorik und Kurswinkelmessung zu einer konsistenten Schätzung von Position, Geschwindigkeit, Lage und Kurs. Im eigenen Stack übernimmt das der Knoten `ekf_node` aus dem Paket `robot_localization`; die Wellenfilterung kommt als eigener Knoten dahinter.

7.1 Aufbau der Filterkette

Bewährt hat sich der in der ROS-Welt übliche Aufbau mit zwei Filterinstanzen. Die erste liefert eine *stetige* Schätzung ohne GNSS – wichtig, weil ein GNSS-Sprung sonst als Geschwindigkeitspuls in die Regelung durchschlägt. Die zweite fusioniert zusätzlich die GNSS-Position und liefert die global richtige Lage. Ein weiterer Knoten rechnet die geografischen Koordinaten in das kartesische Weltsystem um.

- **Erste Instanz (lokal, stetig):** Drehraten und Beschleunigungen aus der IMU, Kurswinkel aus der GNSS-Doppelantenne. Diese Schätzung darf nie springen und wird von der Regelung verwendet.
- **Zweite Instanz (global):** zusätzlich die umgerechnete GNSS-Position. Diese Schätzung darf springen und wird für Pfad und Missionslogik verwendet.
- **Zweidimensionaler Betrieb.** Da nur die Horizontalbewegung geregelt wird, kann der Filter auf die Ebene beschränkt werden. Das verhindert, dass Hub- und Nickbewegung aus dem Seegang in die Positionsschätzung einsickern.

7.2 Was der Filter leisten muss

- **Ausfälle überbrücken.** GNSS setzt kurz aus, die Kamera liefert ein unbrauchbares Bild. Der Filter rechnet mit dem Bewegungsmodell weiter, statt zu springen.
- **Driftgeschwindigkeit schätzen.** Die Differenz zwischen Fahrt durchs Wasser und Fahrt über Grund ist der Versatz durch Wind und Strömung. Diese Größe wird in Kapitel 9 direkt für die Kompensation gebraucht.
- **Konsistenz prüfen.** Sensoren, die einander widersprechen, müssen erkannt und abgewertet werden – nicht gemittelt. Ein Ausreißertest über den Mahalanobis-Abstand ist der übliche Weg.

7.3 Wellenfilterung: der wichtigste Kunstgriff bei Seegang

Die Bewegung des Bootes besteht aus zwei überlagerten Anteilen. Der **niederfrequente** Anteil ist die tatsächliche Fahrt über Grund – das ist die Bahn, die geregelt werden soll. Der **hochfrequente** Anteil ist die Schwingung mit den Wellen; er mittelt sich über eine Wellenperiode heraus und verändert die Bahn praktisch nicht:

$$\psi_{\text{mess}}(t) = \underbrace{\psi_{\text{lf}}(t)}_{\text{Regelgröße}} + \underbrace{\psi_{\text{w}}(t)}_{\text{Wellenanteil}} + \underbrace{n(t)}_{\text{Rauschen}}$$

Wenn die Regelung auf ψ_{w} reagiert, arbeitet sie gegen die Wellen an. Die Folgen sind messbar: permanent wechselnde Motorbefehle, stark erhöhter Strom, Verschleiß an Lagern und Fahrtreglern – und keinerlei Verbesserung der Bahntreue. In der Offshore-Technik ist die Trennung beider Anteile seit den 1970er Jahren Standard.

Wellenfilterung trennt Wellenbewegung von Regelbewegung

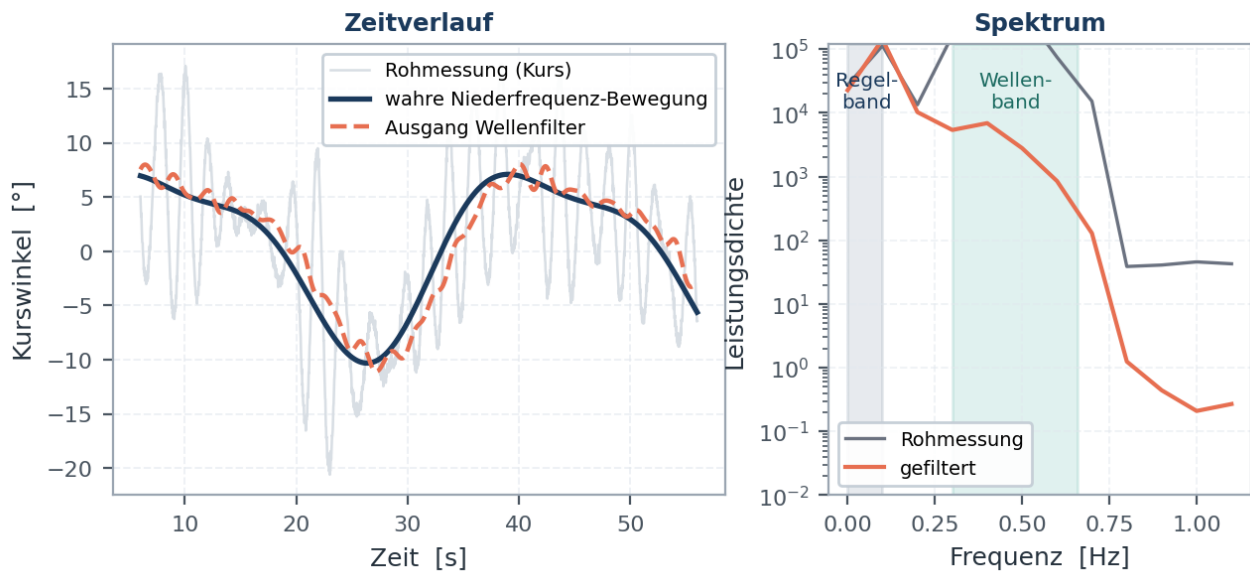


Abbildung 8: Links der Kurswinkel – Rohmessung, wahre niederfrequente Bewegung und Filterausgang. Rechts die zugehörigen Spektren. Der Wellenanteil zwischen etwa 0,3 und 0,7 Hz wird unterdrückt, das Regelband unterhalb 0,1 Hz bleibt erhalten.

Drei Umsetzungsvarianten

Variante	Prinzip	Aufwand	Bewertung
Kerbfiler plus Tiefpass	Bandsperre auf der geschätzten Wellenfrequenz, danach Tiefpass	gering	funktioniert, erzeugt aber Phasenverzug – im Beispiel etwa 1 s. Guter erster Schritt.
Kalman-Filter mit Wellenmodell	das Wellenmodell wird als zusätzlicher Zustand mitgeschätzt und vom Ausgang abgezogen	mittel	kein nennenswerter Phasenverzug, verlangt aber eine Annahme über die Wellenfrequenz
Passiver nichtlinearer Beobachter	verstärkungsgeplanter Beobachter, passt sich dem Seegang an	hoch	Stand der Technik in der dynamischen Positionierung; für dieses Projekt meist über dem Ziel

Das Wellenmodell zweiter Ordnung, das der mittleren Variante zugrunde liegt, lautet im Frequenzbereich:

$$h_w(s) = \frac{2 \lambda \omega_0 \sigma s}{s^2 + 2 \lambda \omega_0 s + \omega_0^2}$$

ω_0	Kreisfrequenz des Wellengipfels, aus dem Spektrum des Nickwinkels bestimmt
λ	Dämpfungsmaß, typisch 0,05 bis 0,20
σ	Intensität, skaliert mit der Wellenhöhe

Phasenverzug ist der Preis – deshalb filtert man an der richtigen Stelle

Ein Tiefpass, der den Wellenanteil unterdrückt, verzögert auch das Nutzsignal. In der schnellen inneren Gierratenschleife wäre das fatal: Dort muss die Regelung sofort reagieren können. Die Trennung erfolgt deshalb nicht im Regler, sondern im Entwurf: Die **innere** Schleife bleibt schnell und ungefiltert, die **äußere** Kursschleife wird bewusst langsam ausgelegt, und die **Positions- und Driftschätzung** wird gefiltert. So folgt das Boot der Welle in der Lage, aber nicht in der Bahn.

Praktische Einstellhinweise

- Bestimmen Sie die Wellenfrequenz aus einer Messfahrt: Zeichnen Sie 30 bis 60 s Nickwinkel auf und bilden Sie das Spektrum. Der Gipfel liegt typisch bei 0,3 bis 0,7 Hz.
- Legen Sie die Bandbreite der äußeren Kursschleife mindestens eine Dekade unter diesen Gipfel, also unter etwa 0,05 Hz.
- Protokollieren Sie den Motorstrom. Steigt er im Seegang deutlich, ohne dass die Fahrt zunimmt, regelt das System gegen die Wellen – dann ist die äußere Schleife noch zu schnell.

8. Wahrnehmung: Bojen und Hindernisse

Wahrnehmungskette von der Kamera zur Objektliste

Robustheit gegen Seegang: Gischt, Spiegelung und Horizontwanken erzeugen Fehldetektionen — nur Objekte mit mehreren Treffern in Folge ('M aus N') werden zu bestätigten Tracks.

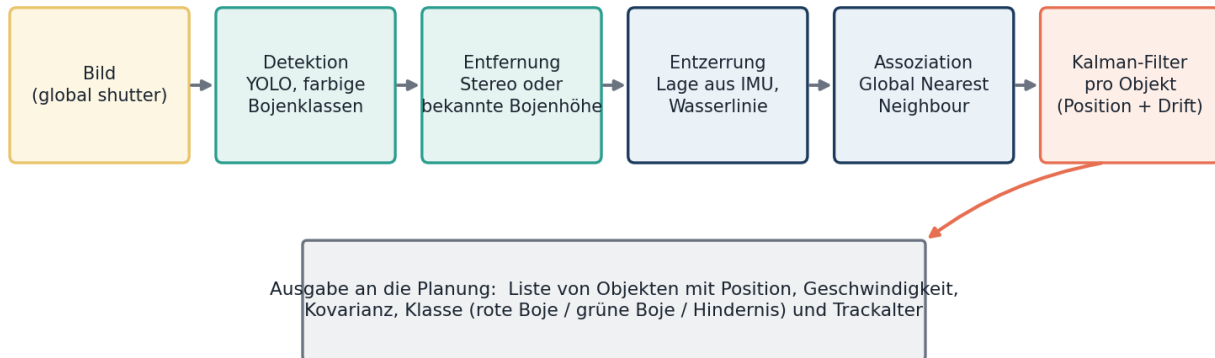


Abbildung 9: Verarbeitungskette von der Kamera bis zur Objektliste. Jede Stufe ist ein eigener Knoten und liefert aufzeichenbare Zwischenergebnisse.

8.1 Erkennung im Bild

Hier ist ein neuronales Netz klar überlegen. Ein Objektdetektor der YOLO-Familie ist der Standard in vergleichbaren Projekten: schnell genug für Echtzeit, robust gegen wechselndes Licht, und mit überschaubarem Aufwand auf eigene Bojen nachtrainierbar. Der klassische Weg über Farbschwellen im HSV-Raum funktioniert im Hallenbad und versagt bei Gegenlicht, Gischt und tiefstehender Sonne.

Datensatz – der Teil, den man unterschätzt

- **Eigene Aufnahmen sind unersetzlich.** Fahren Sie früh eine Aufzeichnungskampagne mit genau der Kamera, die später verbaut ist, und mit genau den Bojen, die später eingesetzt werden. 1.000 bis 2.000 annotierte Bilder reichen für einen brauchbaren Detektor.
- **Schwierige Fälle gezielt sammeln.** Gegenlicht, Wellenkämme, Schaumkronen, Spiegelungen, teilweise verdeckte Bojen, Bojen am Bildrand, Bojen in großer Entfernung.
- **Negativbeispiele einbauen.** Bilder mit Gischt, Möwen, Bojen anderer Kurse und Booten, auf denen nichts annotiert ist. Ohne diese erfindet der Detektor Bojen dort, wo keine sind.
- **Augmentierung.** Spiegelung, Helligkeits- und Farbvariation, leichte Rotation – letztere entspricht genau dem Horizontwanken im Seegang.

8.2 Entfernungsschätzung – die eigentliche Schwierigkeit

Zu wissen, dass im Bild eine grüne Boje ist, nützt wenig. Gebraucht wird ihre Position in Metern. Bei bekannter Bojenhöhe H und Brennweite f in Pixeln gilt:

$$d = \frac{f H}{h_{\text{px}}} \quad \Rightarrow \quad \frac{\partial d}{\partial h_{\text{px}}} = -\frac{f H}{h_{\text{px}}^2} = -\frac{d^2}{f H}$$

Der Fehler wächst also mit dem *Quadrat* der Entfernung. Für die Stereokamera gilt derselbe Zusammenhang mit der Basisbreite B anstelle von H :

$$d = \frac{f B}{\Delta_{\text{px}}}, \quad \sigma_d = \frac{d^2}{f B} \sigma_{\Delta}$$

Beispiel: $f = 800 \text{ px}$, $B = 0,12 \text{ m}$, $d = 15 \text{ m}$ und ein halber Pixel Disparitätsfehler ergeben rund 1,2 m Entfernungsfehler, also etwa 8 %.

Verfahren	Prinzip	Realistische Reichweite	Schwäche
Bekannte Objektgröße	aus der Bildhöhe des Objekts	bis etwa 10-15 m	kleine Bojen belegen bei 20 m nur wenige Pixel
Stereokamera	Disparität zwischen zwei Bildern	bis etwa 15-20 m bei 12 cm Basisbreite	Fehler wächst quadratisch; Wasseroberfläche liefert kaum Textur
Wasserlinien-Geometrie	Winkel unter dem Horizont bei bekannter Kamerahöhe	theoretisch weit	bei 0,25 m Kamerahöhe und 15 m Entfernung unter 1° – im Seegang unbrauchbar

Konsequenz für den Entwurf

Rechnen Sie mit einer verlässlichen Erfassungsreichweite von 15 bis 20 m. Das ist keine Schwäche, sondern eine Entwurfsvorgabe: Die Missionslogik darf immer nur das nächste und das übernächste Tor benötigen. Ein globaler Plan über den gesamten Kurs wäre ohnehin wertlos, weil die Bojen driften. Falls die ungefähre Kurslage vorab bekannt ist, nutzen Sie sie als Vorwissen und die Kamera zur Verfeinerung – das ist deutlich robuster, als beides einzeln zu tun.

8.3 Verfolgung und Zuordnung

Eine Einzelmessung pro Bild reicht nicht. Jede erkannte Boje bekommt einen eigenen Kalman-Filter mit dem Zustand aus Position und Driftgeschwindigkeit und einem Bewegungsmodell konstanter Geschwindigkeit – genau passend für langsam driftende Bojen:

$$\mathbf{x}_k = \begin{pmatrix} p_x \\ p_y \\ \dot{p}_x \\ \dot{p}_y \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Neue Messungen werden den bestehenden Spuren über ein Zuordnungsverfahren zugewiesen. Für die hier auftretende Objektzahl genügt das Nächster-Nachbar-Verfahren mit Mahalanobis-Abstand und einem Ausschlussor:

$$d_M^2 = (\mathbf{z} - \mathbf{H}\hat{\mathbf{x}})^\top \mathbf{S}^{-1} (\mathbf{z} - \mathbf{H}\hat{\mathbf{x}}) < \gamma$$

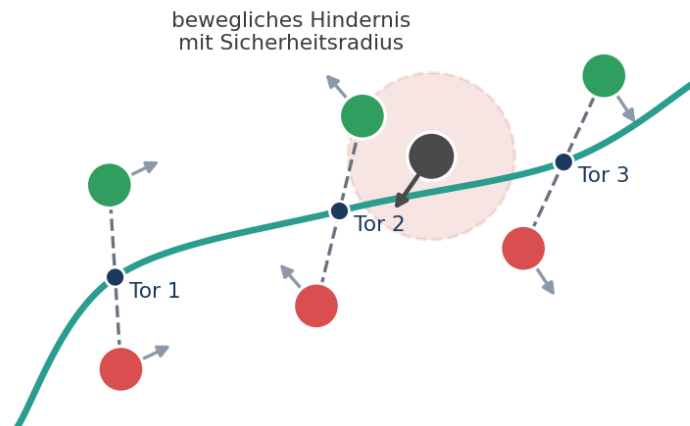
- **Spurbestätigung.** Eine Spur gilt erst nach „M aus N“ Treffern als bestätigt, etwa 3 aus 5. Das unterdrückt Fehldetektionen aus Gischt und Spiegelung, die von Bild zu Bild springen.
- **Spurlöschung.** Nach einigen Sekunden ohne Messung wird die Spur verworfen – aber nicht zu früh: Eine Boje, die hinter einem Wellenkamm verschwindet, ist nach zwei Sekunden wieder da.
- **Farbe ist ein Merkmal, keine Identität.** Nutzen Sie die Klassifikation zur Unterstützung der Zuordnung, nicht als alleiniges Kriterium.

8.4 Vom Bojenpaar zum Pfad

Tore entstehen aus Paaren gegenläufig gefärbter Bojen mit plausiblen Abstand. Der Torzielpunkt ist der Mittelpunkt der Verbindungslinie, der Pfad entsteht durch eine glatte Interpolation zwischen aufeinanderfolgenden Torzielpunkten.

Kursbildung aus beweglichen Bojenpaaren

rote Boje = Backbord · grüne Boje = Steuerbord · graue Pfeile = Bojendrift



Der Sollpfad wird nicht fest abgespeichert, sondern in jedem Zyklus aus den aktuellen Torzentren neu berechnet. Die Bojen sind dabei Messungen mit Unsicherheit – nicht exakte Wegpunkte.

Abbildung 10: Torbildung aus driftenden Bojenpaaren. Der Sollpfad wird in jedem Zyklus neu berechnet, nicht einmalig gespeichert.

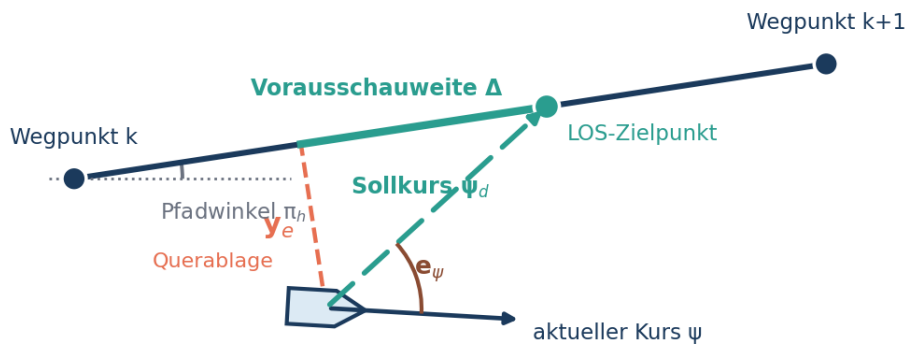
- **Glättung, nicht Ecken.** Eine stückweise gerade Verbindung der Torzentren erzeugt an jedem Tor einen Knick und damit einen Stellgrößensprung. Verwenden Sie eine Spline-Interpolation; sie liefert nebenbei die Krümmung, die im Geschwindigkeitsprofil gebraucht wird.
- **Hysterese beim Torwechsel.** Schalten Sie auf das nächste Tor erst um, wenn die Torlinie tatsächlich überfahren wurde – nicht, wenn der Abstand zum Mittelpunkt einen Schwellwert unterschreitet. Sonst springt die Logik im Seegang hin und her.
- **Unsicherheit berücksichtigen.** Ist die Kovarianz einer Boje groß, verschieben Sie den Zielpunkt zur sichereren Seite und reduzieren Sie die Geschwindigkeit, statt blind auf den geschätzten Mittelpunkt zuzuhalten.

9. Kursführung (Guidance)

Die Kursführung wandelt einen geometrischen Pfad in einen Sollkurs und eine Sollgeschwindigkeit um. Sie ist die Schaltstelle zwischen „wo will ich hin“ und „wie stelle ich das Boot ein“.

9.1 Line-of-Sight als Grundverfahren

Line-of-Sight-Kursführung



$$\psi_d = \pi_h - \arctan(y_e / \Delta)$$

kleines Δ → aggressives Einlenken, Neigung zum Pendeln

großes Δ → ruhiger Lauf, aber langsames Einfangen

Abbildung 11: Geometrie der LOS-Kursführung. Das Boot steuert nicht den Wegpunkt an, sondern einen Punkt auf dem Pfad in der Vorausschauweite Δ .

Das Verfahren ahmt nach, was ein erfahrener Rudergänger tut: Es peilt einen Punkt auf dem Sollpfad an, der ein Stück voraus liegt.

$$\psi_d = \pi_h - \arctan\left(\frac{y_e}{\Delta}\right)$$

π_h	Richtung des Pfadsegments im NED-System
y_e	Querablage vom Pfad, gemessen senkrecht zum Segment
Δ	Vorausschauweite
ψ_d	resultierender Sollkurs

Die Vorausschauweite Δ ist der wichtigste Einstellparameter des gesamten Systems. Ein kleines Δ lenkt aggressiv auf den Pfad zurück, neigt aber zum Pendeln, besonders im Seegang. Ein großes Δ läuft ruhig, fängt den Pfad aber langsam ein. Startwert: das Drei- bis Fünffache der Bootslänge, also 3 bis 5 m. Bei rauem Wasser eher vergrößern.

9.2 Warum einfaches LOS bei Wind nicht genügt

Wind und Strömung versetzen das Boot seitwärts. Das LOS-Gesetz regelt den *Kurswinkel*, nicht den *Kurs über Grund*. Das Boot stellt sich also auf einen Winkel ein, bei dem sich Rückstellwirkung und Versatz gerade aufheben – bei einer von null verschiedenen Querablage. Diese bleibende Ablage verschwindet nicht mit der Zeit.

Ohne Integralanteil bleibt bei Querversatz ein Restfehler

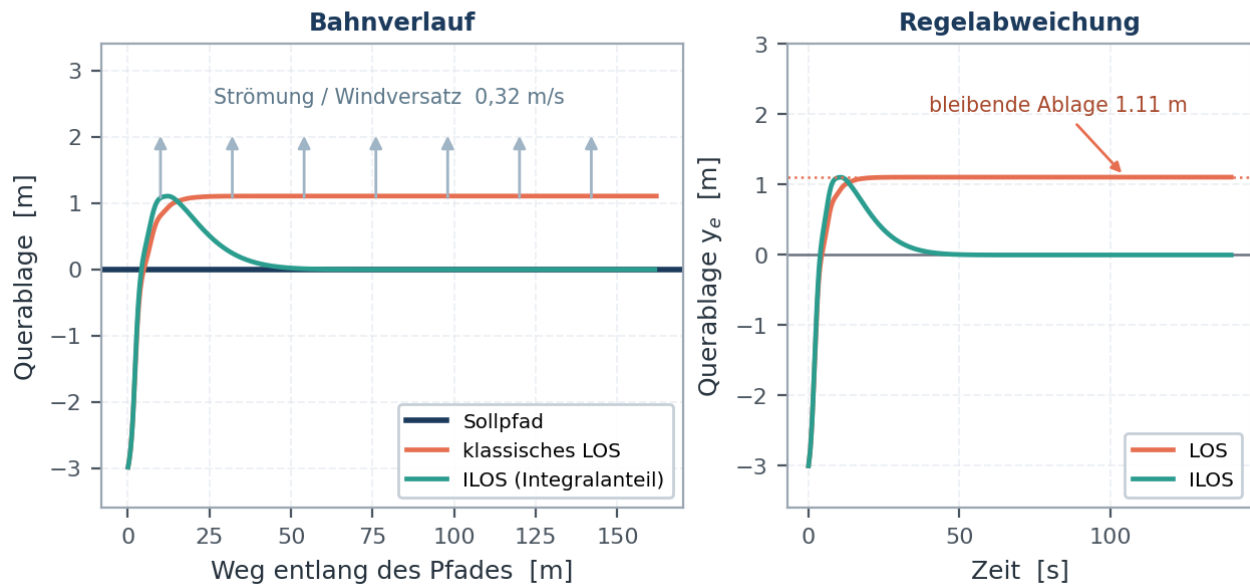


Abbildung 12: Simulation mit 0,32 m/s Querversatz. Das einfache LOS-Gesetz stabilisiert sich bei 1,11 m Ablage, die Integralvariante ILOS läuft auf null.

Bei einem Bojenkurs entscheidet genau dieser Meter darüber, ob ein Tor getroffen oder verfehlt wird. Die Abhilfe besteht darin, den unbekannten Krabbenwinkel mitzuschätzen und aufzuschalten. Das Integral-LOS erweitert das Gesetz um einen Integralzustand y_{int} :

$$\psi_d = \pi_h - \arctan\left(\frac{y_e + \sigma y_{\text{int}}}{\Delta}\right)$$

$$\dot{y}_{\text{int}} = \frac{\Delta y_e}{\Delta^2 + (y_e + \sigma y_{\text{int}})^2}$$

Die Alternative, das adaptive LOS, schätzt den Krabbenwinkel direkt als Parameter und schaltet ihn additiv auf:

$$\psi_d = \pi_h - \hat{\beta} - \arctan\left(\frac{y_e}{\Delta}\right)$$

$$\dot{\hat{\beta}} = \gamma \frac{\Delta y_e}{\sqrt{\Delta^2 + y_e^2}}$$

Verfahren	Idee	Eignung hier
ILOS – Integral-LOS	Integralzustand der Querablage im LOS-Gesetz	einfachste Umsetzung, sehr gut bei nahezu konstantem Versatz; empfohlener Einstieg
ALOS – adaptives LOS	Krabbenwinkel als Parameter geschätzt und additiv aufgeschaltet	besser bei schnell veränderlichem Versatz, etwa in Böen; nachweislich stabil
ELOS – beobachterbasiert	Störgrößenbeobachter schätzt den Krabbenwinkel	gleichwertige Alternative, Auslegung über die Beobachterbandbreite

Konkrete Empfehlung: mit ILOS beginnen, weil es wenige Zeilen Code sind und sofort wirkt. Wenn im Seeversuch sichtbar wird, dass der Versatz in Böen stark schwankt und der Integralanteil hinterherläuft, auf ALOS umstellen. Bei beiden gilt: Der Integral- bzw. Anpassungsanteil muss begrenzt

und beim Wechsel des Pfadsegments zurückgesetzt werden, sonst schleppt das System einen alten Wert in das nächste Segment.

9.3 Umgang mit beweglichen Bojen

- **Pfad neu berechnen, nicht nachziehen.** In jedem Zyklus aus den aktuellen Torzentren, wie in Kapitel 8 beschrieben.
- **Vorhalten.** Wenn eine Boje driftet, zielen Sie nicht auf ihre aktuelle, sondern auf ihre voraussichtliche Position zum Durchfahrtszeitpunkt. Die Driftgeschwindigkeit liefert der Kalman-Filter, die Restfahrzeit ergibt sich aus Abstand und Sollgeschwindigkeit.
- **Stabilität vor Genauigkeit.** Ein leicht geglätteter Zielpunkt, der ruhig steht, ist besser als ein exakter, der bei jedem Bild zappelt. Filtern Sie den Zielpunkt zusätzlich über etwa eine Sekunde.

9.4 Geschwindigkeit ist eine Regelgröße

Ein häufiger Entwurfsfehler ist, die Geschwindigkeit fest vorzugeben. Sie ist die billigste Sicherheitsreserve, die das System hat: Langsamer fahren verschafft Rechenzeit, verkleinert den Bremsweg und verringert die Wirkung von Zustandsfehlern. Bilden Sie die Sollgeschwindigkeit als Minimum mehrerer Grenzen:

$$u_d = \min \left\{ u_{\max}, \sqrt{\frac{a_{\perp, \max}}{\kappa}}, u_{\max} (1 - k_1 \sigma), u_{\max} (1 - k_2 \varrho), u_{\max} (1 - k_3 |e_\psi|) \right\}$$

κ	Krümmung des Sollpfads an der aktuellen Stelle
$a_{\perp, \max}$	zulässige Querbeschleunigung, Startwert 0,8 bis 1,5 m/s ²
σ	Unsicherheitsmaß der Objektschätzung
ϱ	Kollisionsrisiko aus dem SB-MPC, siehe Kapitel 10
e_ψ	Kursabweichung – bei großem Fehler erst drehen, dann Fahrt aufnehmen

Das Tempo ist eine Regelgröße: langsamer heißt sicherer

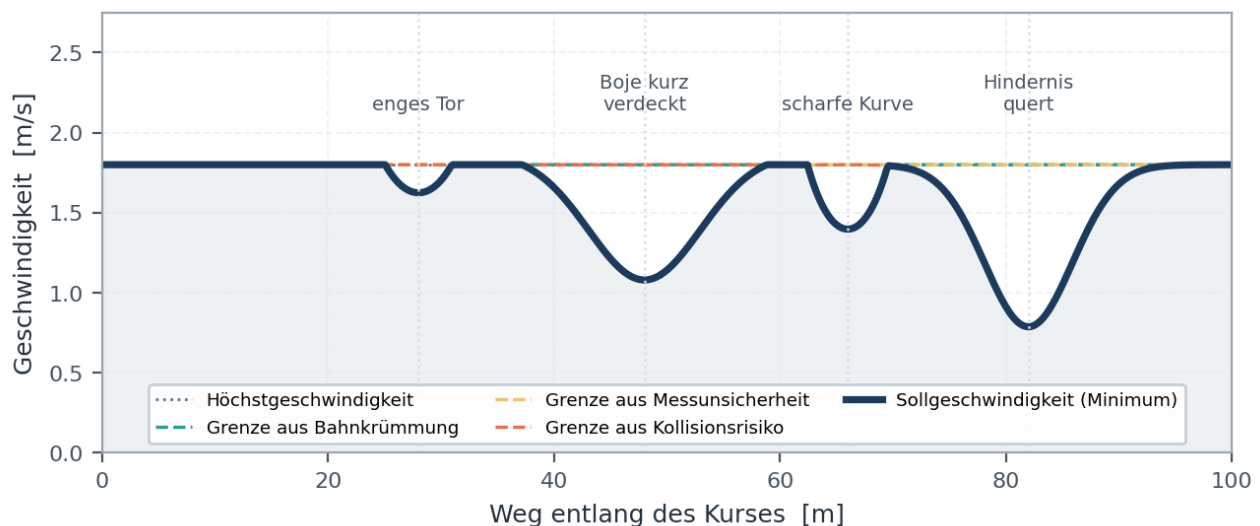


Abbildung 13: Die Sollgeschwindigkeit ergibt sich als Minimum aus Bahnkrümmung, Messunsicherheit, Kollisionsrisiko und Höchstgeschwindigkeit.

10. Kollisionsvermeidung

Diese Schicht darf den Sollkurs der Kursführung überstimmen. Sie arbeitet auf der Objektliste aus Kapitel 8 und liefert zwei Größen: einen Kursversatz und einen Tempofaktor.

10.1 Verfahrensvergleich

Verfahren	Stärke	Schwäche	Urteil
Potentialfelder	sehr einfach, wenige Zeilen	lokale Minima, Aufschwüngen zwischen zwei Hindernissen, berücksichtigt die Fahrzeugdynamik nicht	nicht empfohlen
Dynamic Window (DWA)	berücksichtigt Beschleunigungsgrenzen, bewährt bei Bodenrobotern	kurzer Horizont, mit bewegten Hindernissen nur eingeschränkt brauchbar	als Notbremse denkbar
Velocity Obstacles	sauber auf bewegte Hindernisse zugeschnitten, geometrisch anschaulich	setzt gleichförmige Hindernisbewegung voraus, Fahrzeugdynamik nur näherungsweise	gute Ergänzung, gutes Denkwerkzeug
SB-MPC	endlich viele Verhaltenskandidaten, vorausberechnet und bewertet; echtzeitfähig, nachvollziehbar, in Seerprobungen bewährt	Auswahl der Kandidaten muss durchdacht sein, Kostenfunktion braucht Abstimmung	Empfehlung
Voller NMPC	optimal im Rahmen des Modells, weiche Nebenbedingungen	Lösbarkeit muss garantiert sein, höherer Entwicklungsaufwand	sinnvoller zweiter Ausbauschritt
Bestärkendes Lernen	kann sehr gewandtes Verhalten hervorbringen	kein Sicherheitsnachweis, aufwendige Übertragung aus der Simulation	nur als begrenzter Zusatzanteil, siehe Kapitel 12

10.2 Das Denkwerkzeug: Velocity Obstacles

Velocity Obstacle: verbotene Geschwindigkeiten statt verbotener Orte

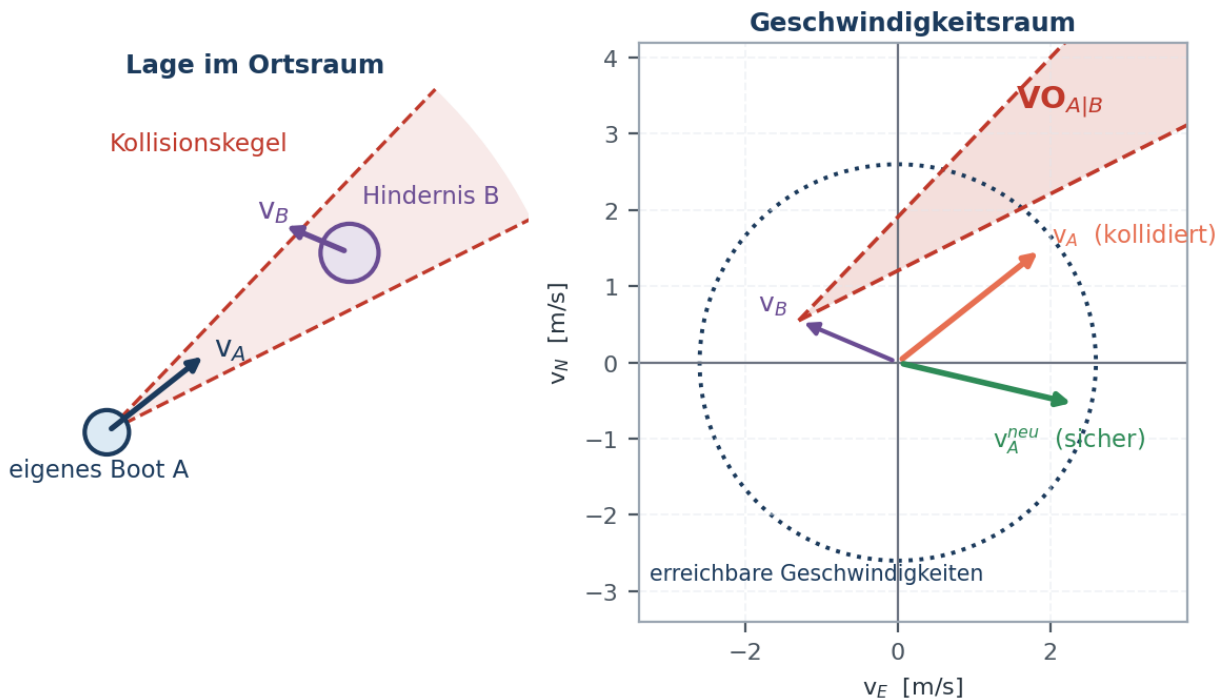


Abbildung 14: Aus dem Kollisionskegel im Ortsraum wird durch Verschiebung um die Hindernisgeschwindigkeit die Menge der verbotenen Eigengeschwindigkeiten.

Die Idee ist elegant: Statt zu fragen, welche *Orte* verboten sind, fragt man, welche *Geschwindigkeiten* verboten sind. Formal ist das Velocity Obstacle die Menge

$$VO_{A|B} = \{ \mathbf{v}_A \mid \exists t > 0 : \|\mathbf{p}_A - \mathbf{p}_B + (\mathbf{v}_A - \mathbf{v}_B)t\| < r_A + r_B \}$$

Alle Geschwindigkeitsvektoren in dieser Menge führen irgendwann zur Kollision. Die Aufgabe reduziert sich darauf, einen erreichbaren Vektor außerhalb zu wählen – möglichst nahe am ursprünglich gewünschten. Auch wenn Sie SB-MPC einsetzen, lohnt sich dieses Bild: Es erklärt, warum ein Ausweichmanöver früh und deutlich sein muss, statt spät und zaghaft.

10.3 Die Empfehlung: szenariobasierter MPC

SB-MPC wählt nicht aus einem kontinuierlichen Stellbereich, sondern aus einer endlichen Menge von Verhaltensweisen. Jede besteht aus einem Kursversatz relativ zum Sollkurs der Kursführung und einem Faktor auf die Sollgeschwindigkeit, der bis zu Rückwärtsfahrt reichen kann. Für jede Kombination wird die Bahn über einen endlichen Horizont vorausberechnet und mit einer Kostenfunktion bewertet; die günstigste wird an die Kursführung weitergegeben. Das Verfahren wurde für kollisionsfreies Fahren großer Schiffe entwickelt und in Seeerprobungen bestätigt.

SB-MPC: 14 Verhaltenskandidaten werden vorausberechnet und bewertet (grün = geringe Kosten, rot = hohe Kosten)

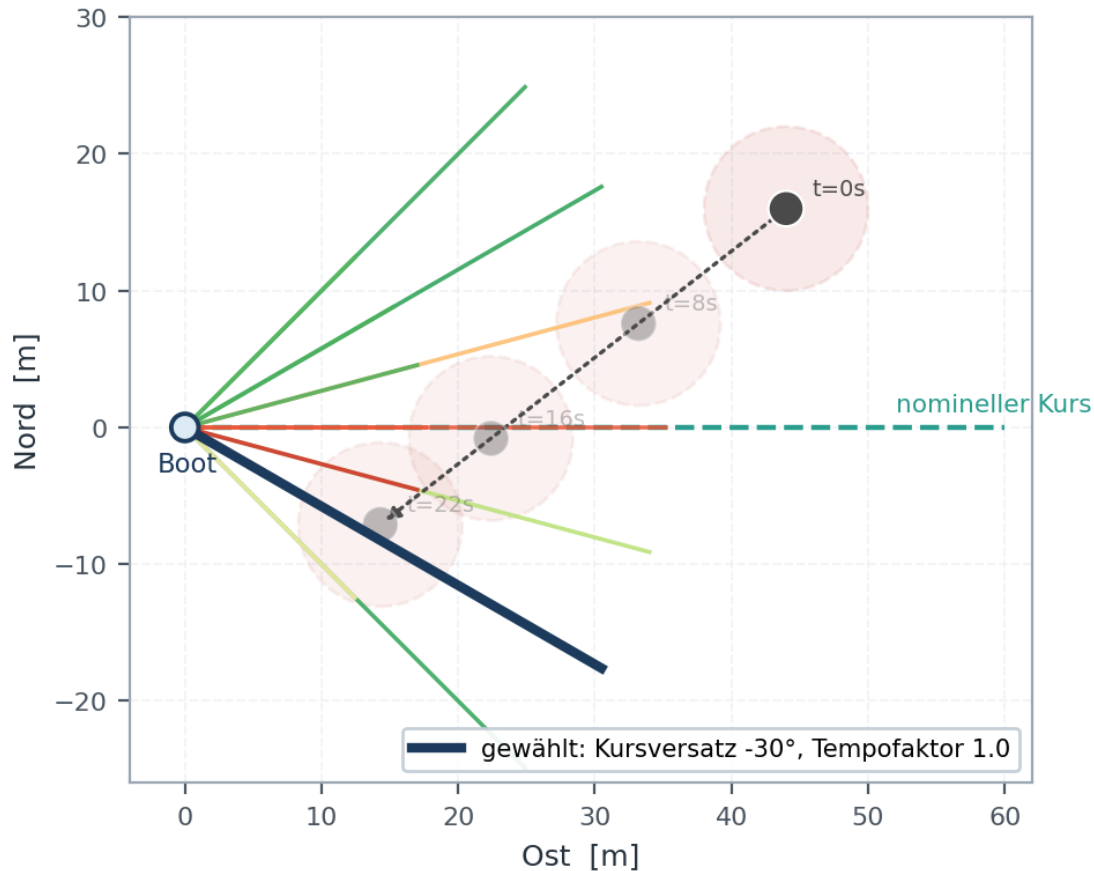


Abbildung 15: Vierzehn Verhaltenskandidaten mit Vorausberechnung über 22 s. Einfärbung nach Kosten. Gewählt wird hier ein Kursversatz von -30° bei voller Fahrt.

Kostenfunktion

$$J(\alpha, f) = w_{\text{koll}} \varrho(d_{\min}) + w_{\alpha} |\alpha| + w_u (1 - f) + w_w \mathbb{I}[(\alpha, f) \neq (\alpha^-, f^-)]$$

mit dem weich modellierten Kollisionsrisiko

$$\varrho(d_{\min}) = \frac{1}{1 + \exp(a (d_{\min} - d_{\text{sicher}}))}$$

α	Kursversatz gegenüber dem Sollkurs der Kursführung
f	Faktor auf die Sollgeschwindigkeit, auch negativ
$d_{\min}$	kleinster Abstand zum Hindernis über den Horizont
(α^-, f^-)	im vorigen Zyklus gewähltes Verhalten

Der letzte Term ist der wichtigste und wird am häufigsten vergessen. Ohne Bestrafung des Verhaltenswechsels springt das System zwischen zwei fast gleichwertigen Manövern hin und her, sobald die Messung leicht rauscht. Genau dieses Zappeln war in der Literatur der Anlass, einen eigenen Übergangskostenterm einzuführen.

Startwerte für die Auslegung

Parameter	Startwert	Bemerkung
Vorausschauhorizont	10–20 s	mindestens das Dreifache der Zeit für ein volles Ausweichmanöver
Zykluszeit	0,1–0,2 s	5 bis 10 Hz reichen völlig
Kursversätze	0, ±15°, ±30°, ±45°, ±60°	symmetrisch; bei Bedarf feiner um null
Tempofaktoren	1,0 / 0,5 / 0,0 / –0,3	Rückwärtsfahrt ist bei zwei Motoren möglich und manchmal die einzige Lösung
Sicherheitsabstand	3 m ruhig, 5–6 m bei Seegang	mindestens das Dreifache der Positionsunsicherheit
Wechselkosten	so groß, dass Zappeln aufhört	im Versuch schrittweise erhöhen, bis das Verhalten ruhig wird

10.4 Sicherheitsabstände bei Seegang

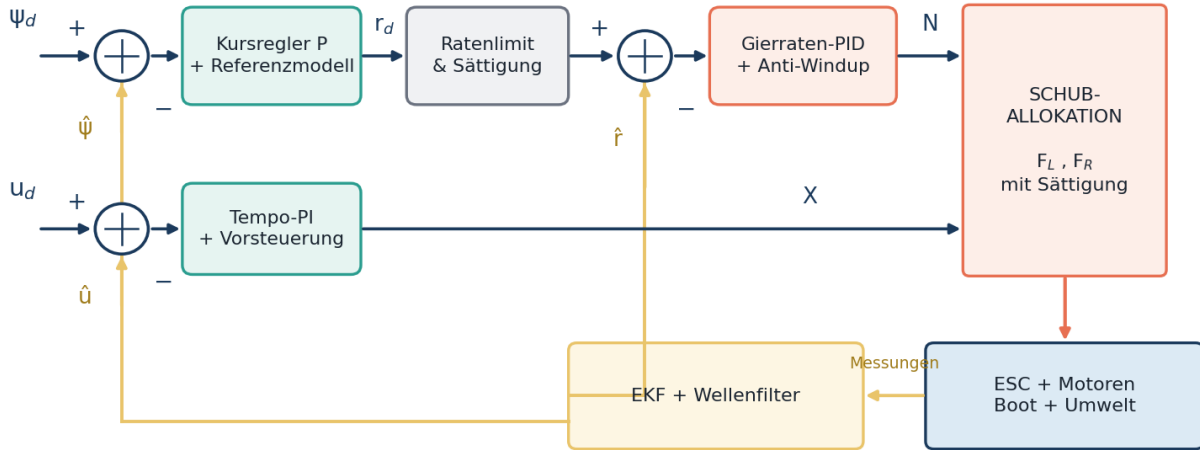
Der Sicherheitsabstand ist keine feste Zahl, sondern muss mit der Unsicherheit wachsen. Im Seegang steigen gleich mehrere Fehlerquellen gleichzeitig: Die Kameramessung wird durch das Wanken ungenauer, die eigene Position schwankt, und die Bahnfolge wird schlechter. Setzen Sie den Radius deshalb zusammengesetzt an:

$$d_{\text{sicher}} = d_0 + 3\sigma_p + c|y_e|$$

Damit wird das System bei rauem Wasser von selbst vorsichtiger, ohne dass jemand einen Schalter umlegen muss.

11. Regelung und Echtzeitschicht

äußere Schleife: Kurs · 10-20 Hz innere Schleife: Gierrate · 50 Hz (mind. 5× schneller)



Kaskadierte Regelungsstruktur

Abbildung 16: Kaskadierte Regelungsstruktur. Die innere Gierratenschleife ist mindestens fünfmal schneller als die äußere Kursschleife.

11.1 Warum eine Kaskade

Ein einzelner PID vom Kursfehler direkt auf das Gierrmoment ist verlockend einfach und funktioniert in ruhigem Wasser. Im Seegang versagt er, weil er keine Möglichkeit hat, die Drehgeschwindigkeit unabhängig zu begrenzen – das Boot wird von den Wellen angedreht, und der Regler weiß nicht, ob die Drehung von ihm selbst oder von außen kommt. Die Kaskade trennt das sauber: Die innere Schleife regelt ausschließlich die Gierrate und wirkt damit unmittelbar gegen jede Störung; die äußere Schleife gibt nur noch eine gewünschte Gierrate vor und darf langsam sein.

11.2 Auslegung der einzelnen Schleifen

Innere Schleife: Gierrate

Reglertyp PID mit Vorsteuerung. Aus dem Nomoto-Modell folgt eine brauchbare Anfangsauslegung. Mit der gewünschten Bandbreite ω_i gilt:

$$K_{ff} = \frac{1}{K}, \quad K_p = \frac{T \omega_i}{K}, \quad K_i = \frac{K_p \omega_i}{5}$$

$$\delta(t) = K_{ff} r_d + K_p e_r + K_i \int e_r dt - K_d \dot{r}, \quad e_r = r_d - \hat{r}$$

Startwert für die Bandbreite: $\omega_i \approx 3 \text{ rad/s}$. Der Differenzialanteil verdient besondere Vorsicht. Er wirkt auf die Ableitung der Gierrate und damit genau auf jenen Frequenzbereich, in dem die Wellen liegen. Beginnen Sie mit $K_d = 0$ und erhöhen Sie ihn nur, wenn die Sprungantwort ohne ihn überschwingt – und filtern Sie ihn dann.

Äußere Schleife: Kurs

Hier genügt ein reiner Proportionalregler, ergänzt um ein Referenzmodell. Das Referenzmodell bildet aus einem Kurssprung eine glatte, mit den Fahrzeuggrenzen verträgliche Sollkurve – damit wird aus einem Zielwechsel kein Stellgrößensprung:

$$r_d = \text{sat}(K_\psi e_\psi, \pm r_{\max}), \quad e_\psi = \text{wrap}_\pi(\psi_c - \hat{\psi})$$

$$\omega_a \approx \frac{\omega_i}{5} \approx 0,6 \text{ rad/s}$$

Die Winkeldifferenz muss zwingend auf das Intervall $(-\pi, \pi]$ zurückgefaltet werden. Ein vergessener Umlauf an dieser Stelle führt dazu, dass das Boot bei einem Kurswechsel über Nord die lange Seite herumdreht – ein Klassiker.

Geschwindigkeitsschleife

PI mit Vorsteuerung aus der bekannten Widerstandskurve. Weil der Zusammenhang zwischen Schub und Fahrt quadratisch ist, sollte die Vorsteuerung diesen Zusammenhang abbilden, sonst muss der Integralanteil ihn ständig nachziehen:

$$X_{\text{ff}}(u_d) = \frac{1}{2} \rho S C_t u_d^2 \quad \text{und} \quad X = X_{\text{ff}} + K_{pu} e_u + K_{iu} \int e_u dt$$

11.3 Anti-Windup, Ratenbegrenzung, stoßfreie Umschaltung

Anti-Windup ist bei diesem System nicht optional. Der Integralanteil läuft sonst voll, während das Boot in einer Welle steht, und überschießt beim Freikommen massiv. Setzen Sie Rückrechnung ein: Die Differenz zwischen gewünschter und tatsächlich gestellter Größe wird auf den Integrator zurückgeführt.

$$\dot{I} = K_i e + \frac{1}{T_t} (u_{\text{sat}} - u)$$

T_t	Rückrechnungszeitkonstante, Startwert etwa $T_t \approx \sqrt{T_i T_d}$
u_{sat}	tatsächlich gestellte Größe nach Begrenzung

- **Ratenbegrenzung** auf die Sollgierrate schützt die Mechanik und verhindert, dass das Boot bei einem Zielwechsel eine Vollkreisdrehung einleitet.
- **Stoßfreie Umschaltung** zwischen den Betriebsarten – manuell, autonom, Rückkehr, Halt. Beim Umschalten werden die Integratoren auf den aktuellen Stellwert initialisiert. Ohne diese Maßnahme gibt es beim Aktivieren der Autonomie einen Ruck, der schon manches Testfahrzeug ins Ufer gesetzt hat. In einem ROS-2-Stack bietet sich dafür der Lebenszyklus eines Knotens an: Der Übergang nach „aktiv“ initialisiert die Zustände.

11.4 Schubsättigung: die Strategie entscheidet

Verhalten bei Schubsättigung: die Wahl der Strategie entscheidet

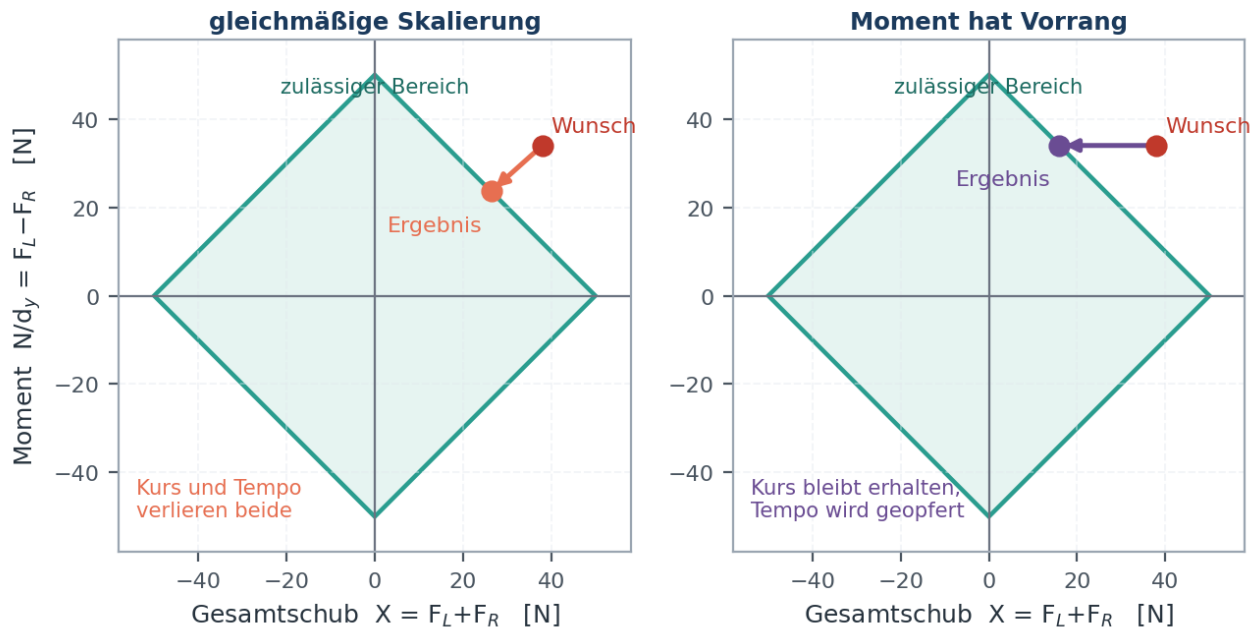


Abbildung 17: Der zulässige Bereich von Gesamtschub und Giermoment ist ein Quadrat auf der Spitze. Liegt der Wunsch außerhalb, muss entschieden werden, was geopfert wird.

Wenn Kurs- und Geschwindigkeitswunsch zusammen mehr verlangen, als die beiden Motoren liefern können, muss die Allokation kürzen. Die naheliegende Lösung – beides gleichmäßig herunterskalieren – ist die falsche: Das Boot verliert Kurs *und* Fahrt und driftet unkontrolliert ab. Richtig ist die Momenten-Priorität: Das geforderte Giermoment wird vollständig gestellt, der Gesamtschub wird auf den verbleibenden Rest reduziert.

$$N_{\text{stell}} = \text{sat}(N, \pm 2 F_{\text{max}} d_y)$$

$$X_{\text{stell}} = \text{sat}\left(X, \pm \left(2 F_{\text{max}} - \frac{|N_{\text{stell}}|}{d_y}\right)\right)$$

Das Boot wird dadurch langsamer, hält aber Kurs. Genau dieses Verhalten brauchen Sie in einer Böe.

11.5 Umsetzung auf dem Mikrocontroller

Die gesamte Kaskade samt Allokation läuft im eigenen Stack auf dem Mikrocontroller, angestoßen von einem Hardware-Timer mit fester Periode. Drei Punkte sind dabei wichtig:

1. **Feste Schrittweite.** Rechnen Sie mit der nominalen Periode, nicht mit der gemessenen Zeitdifferenz. Nur so ist die Integratordynamik reproduzierbar. Wird ein Takt verpasst, wird das als Fehler gezählt und nicht stillschweigend kompensiert.
2. **Keine dynamische Speicherverwaltung im Regelzyklus.** micro-ROS erlaubt einen Betrieb ohne Speicheranforderung zur Laufzeit; nutzen Sie das. Alle Puffer werden beim Start angelegt.
3. **Eigene Zeitüberwachung.** Kommt über eine festgelegte Zeit kein neuer Fahrbefehl vom Linux-Rechner, hält der Mikrocontroller den letzten Kurs für wenige Sekunden und geht danach selbsttätig in den sicheren Zustand. Diese Logik darf nicht auf dem Linux-Rechner liegen – sie muss gerade dann funktionieren, wenn dieser ausfällt.

11.6 Zusätzliche Maßnahmen für den Seegang

Maßnahme	Wirkung
Äußere Schleife langsam auslegen (großes Δ , kleine Verstärkung)	das Boot verfolgt die Bahn, nicht die Wellenbewegung
Totzone auf kleine Kursabweichungen	keine Stellbewegung für Fehler, die ohnehin im Rauschen liegen
Ratenbegrenzung der Motorbefehle	schützt die Fahrtregler und glättet die Stromaufnahme
Geschwindigkeit bei hohem Seegang reduzieren	weniger Schlägen, stabilere Kamerabilder, mehr Rechenzeit
Motorbefehle und Strom aufzeichnen	der Indikator dafür, ob das System gegen die Wellen arbeitet

11.7 Wann sich der Umstieg auf NMPC lohnt

Die Kaskade ist der richtige Einstieg. Ein nichtlinearer modellprädiktiver Regler wird dann interessant, wenn Sie Stellgrößengrenzen und Sicherheitsabstände nicht mehr nachgelagert behandeln, sondern direkt in der Optimierung als Nebenbedingungen führen wollen – dann verschmelzen Kursführung, Kollisionsvermeidung und Regelung zu einem Problem:

$$\min_{\mathbf{u}(\cdot)} \int_t^{t+T_p} \|\mathbf{x}(\tau) - \mathbf{x}_{\text{ref}}(\tau)\|_{\mathbf{Q}}^2 + \|\mathbf{u}(\tau)\|_{\mathbf{R}}^2 d\tau$$

unter den Nebenbedingungen der Fahrzeugdynamik, der Stellgrenzen und der Mindestabstände. Dafür existieren eingebettete Löser, die für genau solche Echtzeitanwendungen ausgelegt sind und auf Kleinrechnern laufen. Planen Sie das als zweiten Ausbauschritt, nachdem die Kaskade auf dem Wasser nachgewiesen ist.

12. Welche Rolle die KI wirklich spielt

Die Aufgabenstellung nennt „KI-Steuerung“. Es lohnt sich, das auseinanderzunehmen, denn die beiden Teilprobleme haben entgegengesetzte Eigenschaften.

Arbeitsteilung zwischen KI und Regelungstechnik

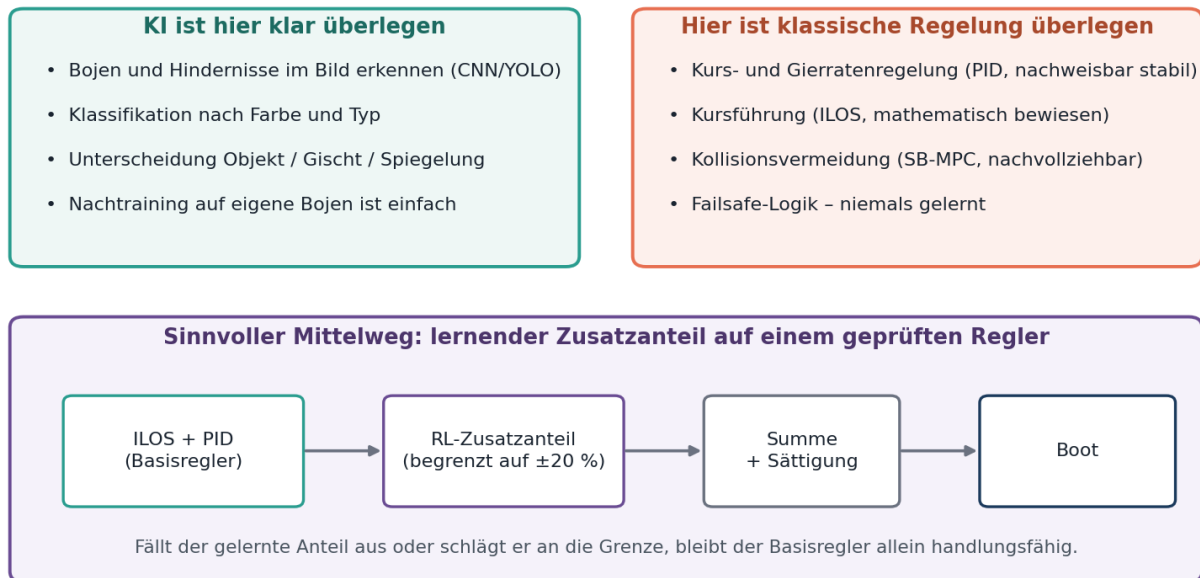


Abbildung 18: Arbeitsteilung. Oben die Domänen, unten die empfohlene hybride Struktur mit begrenztem lernendem Anteil.

12.1 Wahrnehmung: hier gewinnt das Netz

Bojen und Hindernisse in einem Kamerabild mit Gischt, Gegenlicht und wechselndem Horizont zu erkennen, ist ein Musterproblem – genau das, wofür Faltungsnetze gebaut sind. Von Hand programmierte Regeln über Farbe und Form erreichen diese Robustheit nicht. Hier sollten Sie ohne Zögern ein gelerntes Verfahren einsetzen.

12.2 Regelung: hier gewinnt die Mathematik

Für Kursregelung und Kursführung existieren Verfahren mit Stabilitätsnachweis. Ein gelernter Regler kann in der Simulation besser aussehen, aber Sie können nicht zeigen, dass er in einer Seegangssituation, die im Training nicht vorkam, nicht instabil wird. Bei einem Fahrzeug, das an einem Strand oder an einem Badegast landen kann, ist das kein akzeptables Restrisiko.

12.3 Der sinnvolle Mittelweg

Wenn Sie lernende Verfahren in der Regelung einsetzen wollen, dann als **begrenzten Zusatzanteil** auf einem geprüften Basisregler:

$$\delta = \underbrace{\delta_{\text{basis}}}_{\text{ILOS + PID}} + \text{sat}(\delta_{\text{RL}}, \pm 0,2 \delta_{\text{max}})$$

Der Basisregler liefert die Grundstellgröße. Der gelernte Anteil korrigiert das, was das einfache Modell nicht erfasst: Nichtlinearitäten der Propeller, systematische Windwirkung, Wellenmuster. Fällt er aus oder läuft er in die Grenze, bleibt der Basisregler allein handlungsfähig. In einem ROS-2-Stack lässt sich das sauber trennen: Der lernende Anteil ist ein eigener Knoten, dessen Ausfall die Regelung nicht anhält.

Vorgehen, falls Sie diesen Weg gehen

1. Basisregler auf dem Wasser nachweisen. Erst dann beginnt die Arbeit am lernenden Anteil – nicht parallel.
2. In der Simulation trainieren, mit breit gestreuten Parametern: Masse, Schwerpunkt, Motorkennlinien, Wind- und Wellenparameter zufällig variieren. Ohne diese Streuung überträgt sich nichts auf die Realität.
3. Die Begrenzung des Zusatzanteils hart in der Software verankern, nicht nur in der Belohnungsfunktion.
4. Gegen den reinen Basisregler messen. Wenn der Zusatzanteil keinen messbaren Gewinn bringt, lassen Sie ihn weg – er kostet Nachvollziehbarkeit.

Formulierungshilfe für Bericht oder Präsentation

„Das System nutzt maschinelles Lernen für die Umfeldwahrnehmung und modellbasierte Regelungstechnik für Führung und Stabilisierung. Die Kollisionsvermeidung arbeitet szenariobasiert prädiktiv und ist damit in jedem Zyklus erklärbar.“ Das ist fachlich korrekt, trifft den Stand der Technik und ist deutlich belastbarer als die Behauptung, das Boot werde von einer KI gesteuert.

13. Gesamtverhalten in der Simulation

Die folgende Rechnung setzt alle beschriebenen Bausteine zusammen: ILOS-Kursführung mit fortlaufender Pfadneuberechnung aus driftenden Torzentren, SB-MPC-Verhaltensauswahl gegen ein bewegliches Hindernis, kaskadierte Regelung auf Basis des Nomoto-Modells und ein Geschwindigkeitsprofil, das in Kurven und Ausweichmanövern zurücknimmt. Als Störung wirkt ein konstanter Versatz von 0,24 m/s.

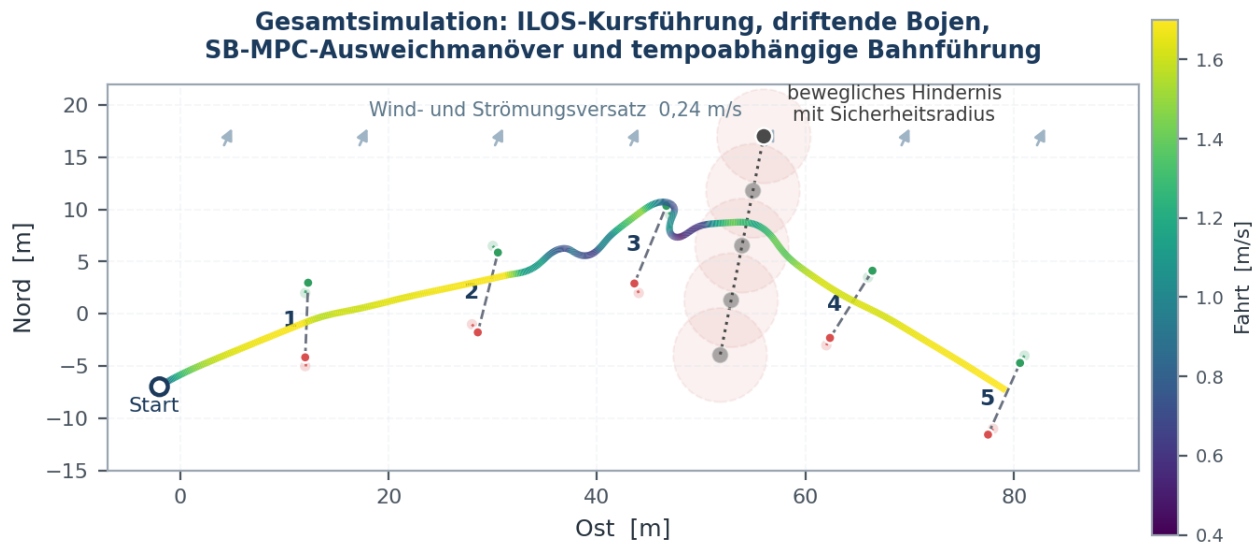


Abbildung 19: Gesamtsimulation. Die blassen Bojen zeigen die Ausgangslage, die vollen die Endlage; die gepunkteten Linien dazwischen sind die Driftwege. Die Bahnfarbe gibt die Fahrt wieder.

Kenngröße	Ergebnis
Passierte Tore	5 von 5
Fahrzeit	69,9 s
Mindestabstand zum beweglichen Hindernis	6,2 m (gefordert: 4,5 m)
Verhalten in den Kurven	selbsttätige Verlangsamung auf etwa 0,6 m/s
Bleibende Querablage trotz Versatz	keine – der Integralanteil wirkt

Zwei Dinge sind an dieser Rechnung wichtig. Erstens ist sie kein Beweis, sondern eine Plausibilitätsprüfung der Architektur: Sie zeigt, dass die Bausteine zusammenpassen und sich nicht gegenseitig blockieren. Zweitens ist das Modell bewusst einfach gehalten – Wellenanregung, Sensorrauschen und Rechenlatenzen fehlen. Genau dafür existieren die Teststufen in Kapitel 14.

Ein Fehler, den diese Rechnung sichtbar gemacht hat

Im ersten Anlauf verfehlte das Boot Tor 3 und lief anschließend geradeaus davon. Ursache war nicht die Regelung, sondern die Torwechsel-Bedingung: Solange sie nur den Abstand zum Torzentrum abfragte, blieb das System nach einem Ausweichmanöver am bereits passierten Tor hängen, und die LOS-Führung zeigte immer weiter geradeaus. Die Abhilfe – den Wechsel am Überfahren der Torlinie festzumachen – steht in Kapitel 8 und ist ein gutes Beispiel dafür, wofür Stufe 0 gut ist.

14. Simulation und Erprobung

Sechs Teststufen von der Gleichung bis zur See

0	Stufe 0 · Modell im Rechner 3-DOF-Modell in Python oder Matlab. Regler, Guidance und Kostenfunktion prüfen.	Abnahme: Sprungantwort stabil, kein Pendeln
1	Stufe 1 · VRX / Gazebo Wind, Wellen, Kamerabild. Kompletter ROS-2-Stack mit echter Software.	Abnahme: Kurs vollständig, kein Kontakt
2	Stufe 2 · SITL und HITL Echte Autopilot-Firmware, echte Rechenzeiten und Busse.	Abnahme: Rechenlast unter 60 %, Latenz < 100 ms
3	Stufe 3 · Trockentest Boot aufgebockt, Propeller frei. Alle Failsafes gezielt auslösen.	Abnahme: Jeder Failsafe greift reproduzierbar
4	Stufe 4 · Stillgewässer Teich oder Hafenbecken, feste Bojen. Identifikation und Reglertuning.	Abnahme: Querablage unter 0,5 m bei Sollfahrt
5	Stufe 5 · Offenes Wasser Wind und Wellen, bewegte Bojen, Begleitboot und Funk-Not-Aus.	Abnahme: Kurs bei Bft 3-4 vollständig gefahren

Keine Stufe überspringen. Jeder Fehler wird zuerst in der niedrigsten Stufe reproduziert, in der er auftritt.

Abbildung 20: Die sechs Teststufen mit jeweils einem schriftlichen Abnahmekriterium.

14.1 Zur Simulationsumgebung

Für Stufe 1 bietet sich VRX an, eine auf Gazebo aufsetzende Simulationsumgebung für Wasserfahrzeuge. Sie bringt Plugins für Wind und Wellen mit, deren Parameter sich einstellen lassen, sowie Modelle von Booten mit zwei Antrieben. Seit Version 3.0 läuft sie standardmäßig auf Gazebo Sim Harmonic mit ROS 2 Jazzy. Für einen eigenen Stack ist das besonders günstig, weil die Schnittstellen dieselben ROS-2-Topics sind wie am realen Boot: Sie tauschen nur die Treiberknoten aus, der Rest des Graphen bleibt unverändert.

Für Stufe 2 tritt an die Stelle der bei fertigen Autopiloten üblichen Software-in-the-Loop-Simulation der Betrieb der echten Mikrocontroller-Firmware am Schreibtisch: Der Mikrocontroller läuft mit der endgültigen Firmware, ist über micro-ROS angebunden, empfängt aber simulierte Zustände und gibt seine Motorbefehle an die Simulation zurück. Damit prüfen Sie genau das, was die Simulation allein nicht abdeckt: Rechenzeiten, Übertragungslatenz und das Verhalten bei verlorenen Nachrichten.

14.2 Was auf jeder Stufe wirklich geprüft wird

Stufe	Prüft im Kern	Typischer Fehler, der hier auffällt
0 – Modell	Logik und Mathematik	Vorzeichenfehler, falsche Umschaltbedingungen, instabile Verstärkungen
1 – Simulation	Zusammenspiel der Knoten	Topics, die niemand abonniert; Zustandsautomaten, die hängen bleiben; falsche Dienstgüte
2 – Firmware am Tisch	Zeitverhalten und Übertragung	Regelschleife hält ihre Rate nicht; Agent verliert die Verbindung; Warteschlangen laufen voll
3 – Trocken	Verkabelung und Sicherheit	vertauschte Motoren, falsches Vorzeichen des Giermoments, Not-Aus ohne Wirkung
4 – Stillwasser	Streckenparameter und Reglerabstimmung	Pendeln um den Pfad, zu aggressive Vorausschauweite, falsche Geschwindigkeitskennlinie
5 – Offenes Wasser	Störfestigkeit	bleibende Querablage, Regelung arbeitet gegen die Wellen, Fehldetektionen durch Gischt

14.3 Aufzeichnung ist keine Kür

Auf dem Wasser lässt sich nichts anhalten und nichts wiederholen. Was nicht aufgezeichnet wurde, ist verloren. Der große Vorteil des eigenen ROS-2-Stacks ist, dass die Aufzeichnung ohne Zusatzarbeit den gesamten Knotengraphen mitschreibt und sich hinterher in Echtzeit oder verlangsamt abspielen lässt – der Stack merkt den Unterschied nicht.

- Zeichnen Sie alle Sensorrohdaten, den geschätzten Zustand, den Sollkurs vor und nach dem Eingriff der Kollisionsvermeidung, das gewählte Verhalten samt Kosten aller Kandidaten sowie Motorbefehle und Ströme auf.
- Kamerabilder mit eingezeichneten Detektionen sind die wertvollste Quelle für das Nachtraining des Detektors – jede Testfahrt vergrößert Ihren Datensatz.
- Legen Sie sich früh ein Auswerteskript an, das aus einer Aufzeichnung automatisch Querablage, Motorstrom pro Meter und Mindestabstände berechnet. Ohne dieses Skript vergleichen Sie Testfahrten nach Gefühl.

15. Sicherheit und Rückfallverhalten

Das Boot fährt auf offenem Wasser, möglicherweise in der Nähe anderer Fahrzeuge und Personen. Die Failsafe-Logik ist deshalb kein Anhang zur Autonomie, sondern deren Voraussetzung. Grundregel: Jeder Ausfall führt in einen *definierten* Zustand, und dieser Zustand wird in Stufe 3 einzeln nachgewiesen. Im eigenen Stack ist zusätzlich wichtig, **wo** die jeweilige Prüfung läuft – eine Überwachung auf dem Linux-Rechner nützt nichts, wenn dieser selbst ausfällt.

Ereignis	Erkennung	Läuft auf	Reaktion
Funkverbindung zur Bodenstation verloren	Zeitüberschreitung, z. B. 5 s	Linux	Mission fortsetzen, Geschwindigkeit halbieren; nach 30 s Rückkehr
Fernsteuerung verloren	Failsafe-Kanal des Empfängers	Mikrocontroller	wie oben; der Not-Aus bleibt die letzte manuelle Instanz
Linux-Rechner antwortet nicht	ausbleibende Fahrbefehle, z. B. 0,5 s	Mikrocontroller	letzten Sollkurs 3 s halten, dann Stopp und Position halten
Einzelner Knoten abgestürzt	Lebenszeichen-Topic bleibt aus	Linux (/watchdog)	Knoten neu starten; bei wiederholtem Ausfall Mission abbrechen
GNSS-Fix verloren oder zu ungenau	Fixgüte und Streuungsmaß	Linux	auf rein relative Messung umstellen, Geschwindigkeit stark reduzieren, Mission abbrechen
Kurswinkel unplausibel	Vergleich von GNSS-Kurs, Drehratenintegration und Kurs über Grund	Linux	auf die verbleibende Quelle umschalten, protokollieren, langsamer fahren
Kamera liefert keine Bilder	Bildalter überschreitet Schwelle	Linux	auf gespeicherte Torpositionen zurückfallen, nach 10 s abbrechen
Akkuspannung unter Schwelle	Spannung unter Last	Mikrocontroller	Rückkehr einleiten; zweite, tiefere Schwelle erzwingt sofortigen Stopp
Wasser im Rumpf	Lecksensor am tiefsten Punkt	Mikrocontroller	sofortige Rückkehr mit voller Fahrt, Warnung an die Bodenstation
Geofence verletzt	Vergleich mit hinterlegtem Polygon	Linux	Stopp oder Rückkehr, je nach Einstellung
Motorstrom dauerhaft zu hoch	gleitender Mittelwert über 10 s	Mikrocontroller	Fremdkörper im Propeller: Stopp, kurzer Rückwärtslauf, danach Abbruch

Die Sicherheitskette darf nicht durch ROS 2 führen

Not-Aus, Stellgrößengrenzen und Stromüberwachung gehören auf den Mikrocontroller und dürfen nicht davon abhängen, dass eine Nachricht rechtzeitig ankommt. DDS ist zuverlässig, aber nicht echtzeitgarantiert. Prüfen Sie in Stufe 3 ausdrücklich den Fall, dass Sie das USB-Kabel zum Linux-Rechner im Betrieb abziehen – das Boot muss dann von selbst in den sicheren Zustand gehen.

15.1 Organisatorische Maßnahmen

- **Begleitboot oder Uferposten** mit Sichtkontakt bei jedem Einsatz auf offenem Wasser. Ein Boot mit leerem Akku treibt ab und ist schnell außer Reichweite.
- **Auftriebskörper und Schleppöse.** Ein geflutetes Boot soll schwimmen und greifbar bleiben.
- **Farbliche Auffälligkeit und Blinklicht.** Ein 1-m-Boot ist im Seegang aus 50 m Entfernung praktisch unsichtbar.
- **Rechtliche Lage vorab klären.** Regeln für unbemannte Wasserfahrzeuge sind je nach Gewässer und Land unterschiedlich; in Hafenbereichen, Naturschutz- und Badezonen gelten oft eigene Bestimmungen. Klären Sie das mit der zuständigen Behörde, bevor Sie auf Stufe 5 gehen.

16. Abstimmung in der richtigen Reihenfolge

Reglerabstimmung ist kein Ausprobieren, sondern eine Reihenfolge. Wer die Kursführung abstimmt, bevor die Gierratenschleife steht, stimmt auf ein bewegliches Ziel ab. Arbeiten Sie diese Liste von oben nach unten ab und gehen Sie erst weiter, wenn der jeweilige Schritt abgenommen ist.

#	Schritt	Was eingestellt wird	Abnahme
1	Aktorik prüfen (trocken)	Drehrichtungen, Vorzeichen von Schub und Giermoment, Neutralstellung und Kalibrierung beider Fahrtregler	Stellbefehl nach Steuerbord dreht den Bug nach Steuerbord
2	Zeitverhalten der Firmware	Timerperiode, Prioritäten, Puffergrößen; Jitter am Oszilloskop oder über einen Prüfausgang messen	50 Hz mit weniger als 1 ms Jitter über 10 min
3	Zustandsschätzung	Sensorkovarianzen, zweidimensionaler Betrieb, Kurswinkel aus zwei Antennen, Antennenabstand korrekt eingetragen	angezeigter Kurs stimmt beim Schieben von Hand auf wenige Grad
4	Systemidentifikation	Zickzack-Versuch bei zwei Geschwindigkeiten, K und T bestimmen	beide Drehrichtungen liefern ähnliche Werte
5	Gierratenschleife	Vorsteuerung zuerst, dann Proportional-, dann Integralanteil; Differenzialanteil zuletzt und nur bei Bedarf	Sprung auf 30°/s in unter 1 s, weniger als 20 % Überschwingen
6	Geschwindigkeitsschleife	Vorsteuerung aus der Schub-Fahrt-Kennlinie, dann PI	Sollfahrt wird in unter 3 s stationär erreicht
7	Kursschleife	Proportionalverstärkung und Referenzmodell	60°-Kurswechsel ohne Pendeln
8	Kursführung	Vorausschauweite Δ , danach Integralanteil des ILOS	gerade Strecke unter 0,5 m Ablage; Restablage bei Seitenwind verschwindet
9	Geschwindigkeitsprofil	Quergrenzbeschleunigung und die drei Reduktionsfaktoren	Kurven werden ohne Ausbrechen durchfahren
10	Wahrnehmung	Erkennungsschwelle, Bestätigungsregel, Filterparameter der Spuren	über eine Minute keine Fehlspur, keine verlorene Boje
11	Kollisionsvermeidung	Sicherheitsradius, Gewichte der Kostenfunktion, zuletzt die Wechselkosten	Ausweichmanöver ist eindeutig, früh und ohne Zappeln
12	Seegangserprobung	Δ vergrößern, äußere Verstärkung senken, Wellenfilter einstellen	Motorstrom steigt im Seegang nicht überproportional

16.1 Werkzeuge, die sich früh lohnen

- **Parameter zur Laufzeit ändern.** Alle Reglerverstärkungen als ROS-2-Parameter führen und über die Standardwerkzeuge im Betrieb verstellen. Wer für jede Änderung neu übersetzen und starten muss, schafft pro Testtag ein Zehntel der Durchläufe.
- **Ein Knoten für Sprungversuche.** Ein kleiner Knoten, der auf Kommando definierte Sprünge auf Gierrate, Kurs oder Fahrt aufschaltet und das Ergebnis aufzeichnet, macht die Abstimmung reproduzierbar.
- **Automatische Auswertung.** Überschwingen, Anstiegszeit und bleibende Abweichung direkt aus der Aufzeichnung berechnen lassen, statt Kurven mit dem Auge zu vergleichen.

Die wirksamste Einzelmaßnahme gegen Pendeln

Wenn das Boot um den Pfad pendelt, ist die erste Maßnahme **nicht**, an den Reglerverstärkungen zu drehen, sondern die Vorausschauweite zu vergrößern. Erhöhen Sie sie in Schritten von 0,5 m, bis das Pendeln aufhört, und nehmen Sie erst dann den Regler wieder vor. Diese Reihenfolge spart erfahrungsgemäß mehrere Testtage.

17. Ablaufplan und Reihenfolge

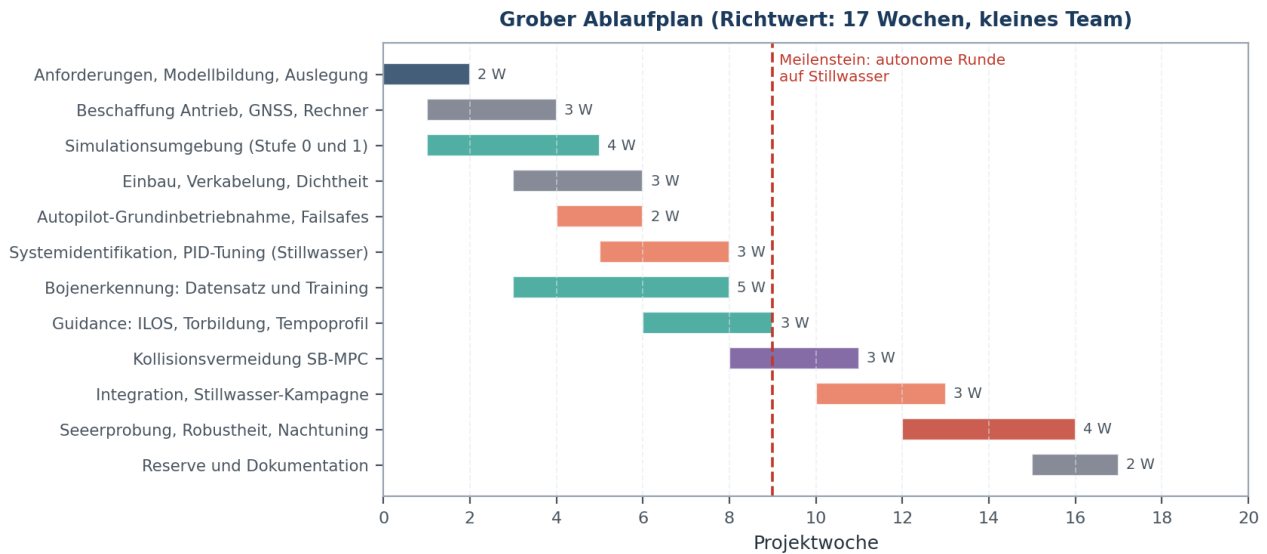


Abbildung 21: Grober Ablauf über 17 Wochen. Die Beschaffung der Sensorik gehört an den Anfang – Lieferzeiten bei GNSS-Empfängern sind die häufigste Verzögerungsursache.

17.1 Was parallel laufen kann und was nicht

- **Parallel:** Simulationsaufbau und mechanische Integration. Die Softwaremannschaft braucht nicht auf das Boot zu warten – wenn der Knotengraph von Anfang an steht, lässt sich jeder Knoten gegen simulierte Eingänge entwickeln.
- **Parallel:** Aufbau des Bilddatensatzes und Reglerabstimmung. Die Aufzeichnungskampagne können Sie auch vom Ufer oder von einem anderen Boot aus fahren.
- **Nicht parallel:** Kollisionsvermeidung vor funktionierender Kursführung. Ohne verlässliche Bahnführung lässt sich nicht unterscheiden, ob ein Fehlverhalten aus der Ausweichlogik oder aus der Regelung stammt.
- **Nicht parallel:** Seeerprobung vor abgeschlossener Stillwasser-Abstimmung. Auf offenem Wasser lässt sich nichts systematisch abstimmen.

17.2 Die vier größten Zeitrissen

Risiko	Gegenmaßnahme
Lieferzeit des GNSS-Empfängers mit zwei Antennen	als Erstes bestellen; als Übergangslösung mit einfachem GNSS plus Kompass arbeiten, aber die Abstimmung nicht darauf aufbauen
Infrastruktur des eigenen Stacks frisst mehr Zeit als erwartet	Knotengraph, Nachrichtentypen, Startskripte und Aufzeichnung in der ersten Projektwoche festlegen und danach nicht mehr umbauen
Wasserdichtigkeit – Nacharbeiten nach jedem Leck	Dichtheitsprüfung mit Unterdruck vor dem ersten Einsatz; Elektronik in ein eigenes, separat abgedichtetes Gehäuse
Wetterfenster für die Seeerprobung	Termine früh und mehrfach ansetzen; Abnahmekriterien so formulieren, dass sie auch bei Bft 3 erreichbar sind

18. Häufige Fehler und wie man sie vermeidet

Fehler	Symptom	Abhilfe
Magnetkompass als einzige Kursquelle	Boot fährt Kreise, sobald die Motoren stark arbeiten	GNSS mit zwei Antennen, Kompass nur als Rückfall
Kursführung ohne Integralanteil	gleichmäßiger Versatz zur Leeseite, Tore werden knapp verfehlt	ILOS oder ALOS einsetzen (Kapitel 9)
Feste Geschwindigkeit	Kurven werden überfahren, Ausweichmanöver kommen zu spät	Geschwindigkeitsprofil nach Abbildung 13
Kein Anti-Windup	starkes Überspringen, nachdem das Boot festgehalten wurde	Rückrechnung auf den Integrator
Torwechsel über den Abstand zum Mittelpunkt	System bleibt an einem Tor hängen oder springt hin und her	Wechsel am Überfahren der Torlinie festmachen, mit Hysterese
Keine Wechselkosten im SB-MPC	Ausweichmanöver wechselt mehrmals pro Sekunde die Richtung	Übergangskostenterm einführen und erhöhen, bis es ruhig wird
Gleichmäßiges Herunterskalieren bei Sättigung	Boot verliert in Böen gleichzeitig Kurs und Fahrt	Momenten-Priorität (Kapitel 11)
Zu kleine Vorausschauweite	Pendeln um den Pfad, besonders im Seegang	Vorausschauweite vergrößern, bevor am Regler gedreht wird
Regelung auf dem Linux-Rechner statt auf dem Mikrocontroller	seltene, aber heftige Stellgrößensprünge; im Labor nicht reproduzierbar	harte Echtzeit auslagern (Kapitel 4)
Voreingestellte Dienstgüte für Kameradaten	Regelung arbeitet auf sekundenalten Bildern	kurze Warteschlange, „best effort“ für Sensordaten
Zeitstempel beim Empfang gesetzt	Zustandsschätzung driftet, Objekte springen	im Treiber mit der Messzeit stempeln, gemeinsame Zeitbasis
Detektor nur auf fremden Daten trainiert	funktioniert im Labor, versagt bei Gegenlicht und Gischt	eigene Aufzeichnungskampagne mit der verbauten Kamera
Teststufen übersprungen	Fehlersuche auf offenem Wasser bei Wellen und knapper Akkuzzeit	Reihenfolge einhalten, jede Stufe schriftlich abnehmen

Anhang A Referenzablauf als Pseudocode

Der folgende Ablauf zeigt eine Iteration der Führungs- und Regelungskette. Der obere Teil läuft auf dem Linux-Rechner, der untere auf dem Mikrocontroller. Fehlerbehandlung ist der Übersicht halber weggelassen.

```
# ===== Linux, /guidance_ilos und /collision_avoidance =====

x      = zustand_aus("/state/filtered")  # Pose, Kurs, Fahrt, Drift
objekte = liste_aus("/objects")         # Tracks mit Kovarianz

# --- Pfad aus den aktuellen Torzentren -----
tore = tore_bilden(objekte)
if torlinie_ueberfahren(x, tore[i]):
    i += 1; ilos_integral = 0.0          # Integrator beim Wechsel leeren
ziel = vorhalten(tore[i], restzeit(x, tore[i]))
pfad = spline(letztes_ziel, ziel, tore[i+1])

# --- Kursfuehrung (ILOS) -----
y_e, pi_h, kappa = pfad.projektion(x.position)
ilos_integral += dt * (D * y_e) / (D**2 + (y_e + sigma*ilos_integral)**2)
ilos_integral = clip(ilos_integral, -I_MAX, +I_MAX)
psi_d         = pi_h - atan2(y_e + sigma*ilos_integral, D)

# --- Geschwindigkeitsgrenzen -----
u_d = min(U_MAX,
          sqrt(A_QUER_MAX / max(kappa, 1e-3)),
          U_MAX * (1 - k1 * unsicherheit(objekte)))

# --- Kollisionsvermeidung (SB-MPC) -----
bestes, j_best = None, INF
for f in TEMPOFAKTOREN:
    for a in KURSVERSAETZE:
        bahn = vorausrechnen(x, psi_d + a, u_d * f, HORIZONT)
        d_min = kleinster_abstand(bahn, objekte.prognose(HORIZONT))
        j      = (W_KOLL * risiko(d_min) + W_KURS * abs(a)
                  + W_TEMPO * (1 - f)
                  + W_WECHSEL * ((a, f) != letztes_verhalten))
        if j < j_best: j_best, bestes = j, (a, f)
a, f = bestes; letztes_verhalten = bestes
veroeffentliche("/cmd/course_safe", psi_d + a, u_d * f)

# ===== Mikrocontroller, /control_node, Timer mit 50 Hz =====

if alter(letzter_befehl) > 0.5:          # Linux-Rechner stumm
    sicherer_zustand(); return

e_psi = wrap_pi(psi_c - x.psi)
u_c   = u_c * clip(1 - k3 * abs(e_psi), 0.35, 1.0)  # erst drehen
r_d   = clip(KP_KURS * e_psi, -R_MAX, +R_MAX)

N = pid_gierrate.schritt(r_d, x.r, DT)  # mit Rueckrechnungs-Antiwindup
X = pi_tempo.schritt(u_c, x.u, DT) + vorsteuerung(u_c)

# --- Schubaufteilung mit Momenten-Prioritaet -----
N_gr = 2 * F_MAX * D_Y
N     = clip(N, -N_gr, +N_gr)
X_gr = 2 * F_MAX - abs(N) / D_Y
X     = clip(X, -X_gr, +X_gr)
F_L   = X/2 + N/(2*D_Y)
F_R   = X/2 - N/(2*D_Y)

if not safety_monitor.ok(): F_L = F_R = 0.0
esc.stellen(F_L, F_R)
```

Anhang B Parameter-Startwerte

Diese Werte gelten für ein Boot von etwa 1 m Länge und 5 bis 8 kg Verdrängung mit zwei Motoren. Sie sind Startpunkte für die Abstimmung, keine Endwerte.

Größe	Symbol	Startwert	Bemerkung
Höchstgeschwindigkeit	$u_{\max}$	1,8 m/s	über der Rumpfgeschwindigkeit nur mit Gleit- oder Katamaranrumpf sinnvoll
Marschgeschwindigkeit	u_d	1,2–1,5 m/s	bei Seegang weiter reduzieren
Halber Motorabstand	d_y	0,12–0,18 m	am Boot ausmessen
Maximalschub je Motor	$F_{\max}$	25–50 N	in der Software auf 60–70 % begrenzen
Nomoto-Verstärkung	K	0,8–1,5 s ⁻¹	aus dem Zickzack-Versuch
Nomoto-Zeitkonstante	T	0,8–1,5 s	aus dem Zickzack-Versuch
Vorausschauweite	Δ	3–5 m	drei- bis fünffache Bootslänge; im Seegang vergrößern
ILOS-Anpassungsfaktor	σ	0,2–0,4	zu groß führt zu Überspringen
Grenze des Integralanteils	$I_{\max}$	entspricht 25° Krabbenwinkel	begrenzen und beim Segmentwechsel zurücksetzen
Bandbreite innere Schleife	ω_i	2–4 rad/s	so schnell wie stabil möglich
Bandbreite äußere Schleife	ω_a	0,4–0,8 rad/s	mindestens Faktor fünf unter der inneren
Maximale Gierrate	$r_{\max}$	40–60 °/s	Ratenbegrenzung setzen
Quergrenzbeschleunigung	$a_{\perp, \max}$	0,8–1,5 m/s ²	bestimmt die Verlangsamung in Kurven
SB-MPC-Horizont	T_p	10–20 s	
SB-MPC-Rate	—	5–10 Hz	
Sicherheitsradius	d_{sicher}	3 m / 5–6 m	ruhiges bzw. bewegtes Wasser
Bestätigungsregel Spuren	M aus N	3 aus 5	
Spurverfall	—	2–4 s ohne Messung	
Wellenband der Filterung	$\omega_0/2\pi$	0,3–0,7 Hz	aus dem Spektrum des Nickwinkels bestimmen
Regelperiode Mikrocontroller	Δt	20 ms	fest, nicht gemessen; Jitter unter 1 ms
Zeitüberschreitung Fahrbefehl	—	0,5 s	danach letzten Kurs halten, nach 3 s Stopp

Anhang C Checkliste vor jedem Einsatz

Vor dem Wassern

- Akku geladen, Spannung unter Last geprüft, Kapazität für die geplante Fahrzeit ausreichend
- Gehäuse geschlossen, Durchführungen fest, Dichtheitsprüfung nicht älter als der letzte Eingriff
- Propeller frei, Befestigung fest, keine Fremdkörper
- Not-Aus mechanisch erreichbar und geprüft
- Geofence für das heutige Gewässer eingetragen und kontrolliert
- Aufzeichnung aktiv, freier Speicherplatz geprüft
- Firmware-Stand des Mikrocontrollers und Parametersatz notiert – ohne diese Notiz ist die Aufzeichnung später nur halb so viel wert

Im Wasser, vor dem Freigeben der Autonomie

- GNSS mit fester Lösung, Kurswinkel aus zwei Antennen gültig
- Angezeigter Kurs stimmt mit der tatsächlichen Ausrichtung überein
- Alle erwarteten Knoten laufen, keine Fehlermeldungen in den Diagnosedaten
- Manuelle Fahrt vorwärts, rückwärts und beide Drehrichtungen geprüft
- Funkstrecke auf der geplanten Maximalentfernung getestet
- Rückfallverhalten einmal ausgelöst und zurückgenommen, einschließlich Abziehen der Verbindung zum Mikrocontroller
- Begleitboot oder Uferposten in Position, Bergungsmittel bereit
- Wind und Seegang gemessen und mit dem Auslegungsfenster verglichen

Nach der Fahrt

- Aufzeichnung gesichert und beschriftet (Datum, Wind, Seegang, Parameterstand)
- Auffälligkeiten unmittelbar notiert – nicht am Abend aus dem Gedächtnis
- Kamerabilder mit Detektionen gesichtet, Fehlerfälle für das Nachtraining markiert
- Auswerteskript über die Aufzeichnung laufen lassen und die Kennzahlen mit der letzten Fahrt vergleichen
- Motoren gespült, Boot getrocknet, Akku auf Lagerspannung gebracht

Anhang D Verwendete Quellen und Literatur

Die folgenden Arbeiten und Dokumentationen bilden die fachliche Grundlage der hier gegebenen Empfehlungen.

Modellbildung, Kursführung und Regelung

- T. I. Fossen: *Handbook of Marine Craft Hydrodynamics and Motion Control*. Standardwerk zu 3-DOF-Modellen, LOS-Kursführung, Zustandsschätzung und Trägheitsnavigation für Schiffe und unbemannte Fahrzeuge.
- T. I. Fossen: *An Adaptive Line-of-Sight (ALOS) Guidance Law for Path Following of Aircraft and Marine Craft*. IEEE Transactions on Control Systems Technology, Bd. 31 (2023). Kompensation von Driftkräften aus Wind, Wellen und Strömung; Vergleich mit ILOS und ELOS.
- W. Caharija u. a.: *Integral Line-of-Sight Guidance and Control of Underactuated Marine Vehicles: Theory, Simulations, and Experiments*. IEEE TCST, Bd. 24 (2016). Grundlage des in Kapitel 9 verwendeten ILOS-Gesetzes.
- A. M. Lekkas, T. I. Fossen: *Integral LOS Path Following for Curved Paths*. IEEE TCST, Bd. 22 (2014). Erweiterung auf gekrümmte Pfade und zeitveränderliche Vorausschauweite.

Wellenfilterung und Zustandsschätzung

- T. Lauvdal, T. I. Fossen: *Robust adaptive ship autopilot with wave filter and integral action*. International Journal of Adaptive Control and Signal Processing, Bd. 12 (1998). Kerbfilter gegen Wellenanteile erster Ordnung im Kurswinkel.
- T. I. Fossen u. a.: Arbeiten zur Wellenfilterung in Systemen der dynamischen Positionierung; Vergleich von Kerbfilter-, beobachterbasierten und passivitätsbasierten Entwürfen.
- J. G. Balchen u. a.: *Dynamic positioning using Kalman filtering and optimal control theory* (1976/1980). Ursprung der Trennung von nieder- und hochfrequenter Bewegung.
- *robot_localization* – Paketdokumentation zu den Zustandsschätzknoten (EKF und UKF) und zur Umrechnung von GNSS-Koordinaten in das kartesische Weltsystem; beschreibt auch den zweistufigen Aufbau aus lokaler und globaler Schätzung.

Kollisionsvermeidung

- T. A. Johansen, T. Perez, A. Cristofaro: *Ship Collision Avoidance and COLREGS Compliance Using Simulation-Based Control Behavior Selection With Predictive Hazard Assessment*. IEEE Transactions on Intelligent Transportation Systems, Bd. 17 (2016). Ursprungsarbeit zum szenariobasierten MPC.
- I. B. Hagen, D. K. M. Kufoalor, T. A. Johansen, E. F. Brekke: Arbeiten zu SB-MPC mit mehreren Entscheidungsschritten und zu Seeerprobungen; Einführung eines Übergangskostenterms gegen oszillierendes Verhalten.
- D. K. M. Kufoalor, E. F. Brekke, T. A. Johansen: *Proactive collision avoidance for ASVs using a dynamic reciprocal velocity obstacles method*. IEEE/RSJ IROS (2018).
- P. Fiorini, Z. Shiller: Ursprungsarbeiten zu Velocity Obstacles; Grundlage von Abbildung 14.

ROS 2, Echtzeit und eingebettete Umsetzung

- *micro-ROS* – Projektdokumentation. Aufbau aus Client auf dem Mikrocontroller und Agent auf dem Linux-Rechner, Kopplung über DDS-XRCE, Unterstützung gängiger 32-Bit-Mikrocontroller und Echtzeitbetriebssysteme, Betrieb ohne dynamische Speicherverwaltung.
- *micro-ROS-Agent* – ROS-2-Paket, das den Mikrocontroller als regulären Teilnehmer in den DDS-Datenraum einbindet.
- *ros2_control*, Dokumentation des Controller Managers. Empfehlungen zur Verringerung der Laufzeitschwankung: SCHED_FIFO mit hoher Priorität, Rechtevergabe über eine Echtzeitgruppe, Einsatz eines Echtzeit- oder Low-Latency-Kerns.

- Übersichtsarbeiten zur Echtzeitfähigkeit von ROS 2 und zum Verhalten der Executor-Implementierungen, einschließlich des statisch geordneten Executors auf der Mikrocontroller-Seite.

Wahrnehmung, Simulation und Praxis

- B. Bingham u. a.: *Toward Maritime Robotic Simulation in Gazebo*. OCEANS 2019. Beschreibung der VRX-Umgebung mit Wind- und Wellen-Plugins.
- VRX-Projektdokumentation. Ab Version 3.0 Betrieb auf Gazebo Sim Harmonic mit ROS 2 Jazzy.
- Technische Berichte von RoboBoat-Teams (u. a. Georgia Tech, AGH Solar Boat, SimLE SeaSentinel, Navier USN). Praxisnahe Darstellungen von YOLO-basierter Bojenerkennung, Sensorfusion, ROS-2-Architekturen und der Diskrepanz zwischen Simulation und Wasser.
- Blue Robotics: Technische Daten des T200-Antriebs (Schub, Strom und Leistung bei 12, 16 und 20 V).
- acados – quelloffenes Rahmenwerk für eingebettete nichtlineare optimale Regelung; Option für den in Kapitel 11 beschriebenen NMPC-Ausbauschritt.

Die Abbildungen 6, 8, 12, 13, 15 und 19 beruhen auf eigenen Simulations- und Modellrechnungen mit den in diesem Dokument angegebenen Parametern. Abbildung 1 verwendet die fetchbegrenzte JONSWAP-Näherung. Die übrigen Abbildungen sind schematische Darstellungen. Alle Formeln sind in Computer Modern gesetzt.